

```
plmdebug /enableDebug [PackageFullName] ""C:\Program Files\Debugging Tools  
for Windows (x64)\WinDbg.exe\" -server npipe:pipe=test"
```

In contrast to Visual Studio, most of the core functionality of WinDbg relies on providing commands to the command window. The provided commands allow you to view execution state, investigate user mode crash dumps, and debug in a variety of modes.

One of the most popular commands in WinDbg is `!analyze -v`, which is used to retrieve a verbose amount of information about the current exception, including:

- FAULTING_IP: instruction pointer at the time of fault
- EXCEPTION_RECORD: address, code, and flags of the current exception
- STACK_TEXT: stack trace prior to exception

For a complete list of all WinDbg commands, see [Debugger Commands](#).

Related topics

- [Testing and debugging tools for Process Lifetime Management \(PLM\)](#)
- [Debugging, testing, and performance](#)

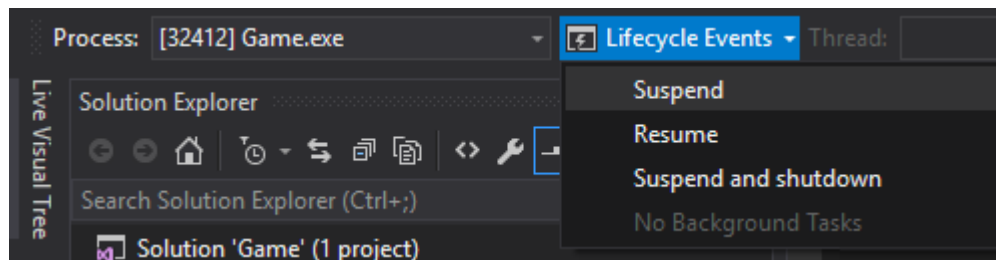
Testing and debugging tools for Process Lifetime Management (PLM)

Article • 10/20/2022

One of the key differences between UWP apps and traditional desktop applications is that UWP titles reside in an app container subject to Process Lifecycle Management (PLM). UWP apps can be suspended, resumed, or terminated across all platforms by the Runtime Broker service, and there are dedicated tools for you to use to force those transitions when you are testing or debugging the code that handles them.

Features in Visual Studio 2015

The built-in debugger in Visual Studio 2015 can help you investigate potential issues when using UWP-exclusive features. You can force your application into different PLM states by using the **Lifecycle Events** toolbar, which becomes visible when you run and debug your title.



The PLMDebug tool

PLMDebug.exe is a command-line tool that allows you to control the PLM state of an application package, and is shipped as part of the Windows SDK. After it is installed, the tool resides in *C:\Program Files (x86)\Windows Kits\10\Debuggers\x64* by default.

PLMDebug also allows you to disable PLM for any installed app package, which is necessary for some debuggers. Disabling PLM prevents the Runtime Broker service from terminating your app before you have a chance to debug. To disable PLM, use the **/enableDebug** switch, followed by the *full package name* of your UWP app (the short name, package family name, or AUMID of a package will not work):

Windows Command Prompt

```
plmdebug /enableDebug [PackageFullName]
```

After deploying your UWP app from Visual Studio, the full package name is displayed in the output window. Alternatively, you can also retrieve the full package name by running **Get-AppxPackage** in a PowerShell console.

```
Name           : 279f7062-ce35-40e8-a69f-cc22c08e0bb8
Publisher      : CN=csang
Architecture   : X86
ResourceId     :
Version        : 1.0.0.0
PackageFullName : 279f7062-ce35-40e8-a69f-cc22c08e0bb8_1.0.0.0_x86__c6sq6kwgxxfcg
InstallLocation : C:\Users\csang\Documents\Visual Studio 2015\Projects\Game\Debug\Game\AppX
IsFramework    : False
PackageFamilyName : 279f7062-ce35-40e8-a69f-cc22c08e0bb8_c6sq6kwgxxfcg
PublisherId    : c6sq6kwgxxfcg
IsResourcePackage : False
IsBundle       : False
IsDevelopmentMode : True
Dependencies    : {Microsoft.VCLibs.140.00.Debug_14.0.23918.0_x86__8wekyb3d8bbwe}
```

Optionally, you can specify an absolute path to a debugger that will automatically launch when your app package is activated. If you wish to do this using Visual Studio, you'll need to specify VSJITDebugger.exe as the debugger. However, VSJITDebugger.exe requires that you specify the "-p" switch, along with the process ID (PID) of the UWP app. Because it's not possible to know the PID of your UWP app beforehand, this scenario is not possible out of the box.

You can work around this limitation by writing a script or tool that identifies your game's process, and then the shell runs VSJITDebugger.exe, passing in the PID of your UWP app. The following C# code sample illustrates a straightforward approach to accomplish this.

C#

```
using System.Diagnostics;

namespace VSJITLauncher
{
    class Program
    {
        static void Main(string[] args)
        {
            // Name of UWP process, which can be retrieved via Task Manager.
            Process[] processes = Process.GetProcessesByName(args[0]);

            // Get PID of most recent instance
            // Note the highest PID is arbitrary. Windows may recycle or
            // wrap the PID at any time.
            int highestId = 0;
            foreach (Process detectedProcess in processes)
            {
                if (detectedProcess.Id > highestId)
                {
                    highestId = detectedProcess.Id;
                }
            }

            // Launch VSJITDebugger.exe, which resides in
```

```

C:\Windows\System32
    ProcessStartInfo startInfo = new
ProcessStartInfo("vsjitdebugger.exe", "-p " + highestId);
    startInfo.UseShellExecute = true;

    Process process = new Process();
    process.StartInfo = startInfo;
    process.Start();
}
}
}

```

Example usage of this in conjunction with PLMDebug:

Windows Command Prompt

```

plmdebug /enableDebug 279f7062-ce35-40e8-a69f-
cc22c08e0bb8_1.0.0.0_x86__c6sq6kwgxxfcg "\"C:\VSJITLauncher.exe\" Game"

```

where `Game` is the process name, and `279f7062-ce35-40e8-a69f-cc22c08e0bb8_1.0.0.0_x86__c6sq6kwgxxfcg` is the full package name of the example UWP app package.

Note that every call to `/enableDebug` must be later coupled to another PLMDebug call with the `/disableDebug` switch. Furthermore, the path to a debugger must be absolute (relative paths are not supported).

Related topics

- [Deploying and debugging UWP apps](#)
- [Debugging, testing, and performance](#)

Test with the Microsoft Emulator for Windows 10 Mobile

Article • 06/13/2023

Simulate real-world interaction with a device and test the features of your app by using the tools included with Microsoft Emulator for Windows 10 Mobile. The emulator is a desktop application that emulates a mobile device running Windows 10. It provides a virtualized environment in which you can debug and test Windows apps without a physical device. It also provides an isolated environment for your application prototypes.

The emulator is designed to provide comparable performance to an actual device. Before you publish your app to the Microsoft Store, however, we recommend that you test your app on a physical device.

You can test your universal app using a unique Windows 10 Mobile emulator image for various screen resolution and screen size configurations. You can simulate real-world interaction with a device and test various features of your app by using the tools included in the Microsoft Emulator.

System requirements

Your computer must meet the following requirements:

CPU (these settings, if available, can be enabled in the BIOS)

- Hardware-assisted virtualization.
- Second Level Address Translation (SLAT).
- Hardware-based Data Execution Prevention (DEP).

RAM

- 4 GB or more.

Operating system

- Windows 10 or later
- 64-bit
- Pro edition or later

See [Install Hyper-V on Windows 10](#).

To check requirements for RAM and operating system, in Control Panel, select **System and Security**, and then select **System**.

Microsoft Emulator for Windows 10 Mobile requires Visual Studio 2015 or later; it is not backward compatible with earlier versions of Visual Studio.

Microsoft Emulator for Windows 10 Mobile cannot load apps that target the Windows Phone OS version earlier than Windows Phone OS 7.1.

Installing, uninstalling, and running the emulator

Installing

[Windows 10 Mobile emulator](#) ships as part of the Windows 10 SDK. The Windows 10 SDK and emulator can be installed as part of the Visual Studio installation. See the [Visual Studio download page](#).

Uninstalling

You can uninstall the Microsoft Emulator for Windows 10 Mobile using Visual Studio setup/repair. Or you can use **Programs and Features** under **Control Panel** to remove the emulator.

When you uninstall the Microsoft Emulator for Windows 10 Mobile, the Hyper-V Virtual Ethernet Adapter that was created for the emulator to use is not automatically removed. You can manually remove this virtual adapter from **Network Connections** in **Control Panel**.

Running

To run an app in the emulator, simply select an emulated device from the **run** drop-down menu in Visual Studio.



What's new in Microsoft Emulator for Windows 10 Mobile

In addition to providing support for Universal Windows Platform (UWP), the emulator has added the following functionality:

- Mouse input mode support to differentiate between mouse and single touch input.
- NFC Support. The emulator allows you to simulate NFC and make it possible to test and develop NFC/Proximity-enabled universal apps.
- Native hardware acceleration improves graphics performance in the emulator by using the local graphics card. You must have a supported graphics card installed, and enable acceleration on the **Sensors** tab of the emulator's **Additional Tools** settings user interface in order to use acceleration.

Features that you can test in the emulator

In addition to the new features mentioned in the previous section, you can test the following commonly used features in the Microsoft Emulator for Windows 10 Mobile.

- **Screen resolution, screen size, and memory.** Reach a broad market for your app by testing it on various emulator images to simulate various screen resolutions, physical sizes, and memory constraints.



- **Screen configuration.** Change the emulator from portrait to landscape mode. Change the zoom setting to fit the emulator to your desktop screen.
- **Networking.** Networking support is integrated with Windows Phone Emulator. Networking is enabled by default. You do not have to install network drivers for Windows Phone Emulator or configure networking options manually in most environments.

The emulator uses the network connection of the host computer. It does not appear as a separate device on the network.

- **Language and region settings.** Prepare your app for an international market by changing the display language and region settings in Windows Phone Emulator.

On the running emulator, go to the **Settings** app, then select the **system** settings, then select **language** or **region**. Change the settings that you want to test. If you're prompted, click **restart phone** to apply the new settings and restart the emulator.

- **Application lifecycle and tombstoning.** Test the behavior of your app when it's deactivated or tombstoned by changing the value of the option **Tombstone upon deactivation while debugging** on the **Debug** page of project properties.
- **Local folder storage (previously known as isolated storage).** Data in isolated storage persists while the emulator is running, but is lost once the emulator closes.
- **Microphone.** Requires and uses the microphone on the host computer.
- **Phone keyboard.** The emulator supports mapping of the hardware keyboard on your development computer to the keyboard on a Windows Phone. The behavior of the keys is the same as on a Windows Phone device
- **Lock screen.** With the emulator open, press F12 on your computer keyboard twice. The F12 key emulates the power button on the phone. The first key press turns off the display. The second key press turns the display on again with the lock screen engaged. Unlock the screen by using the mouse to slide the lock screen upward.

Features that you can't test in the emulator

Test the following features only on a physical device.

- Compass
- Gyroscope
- Vibration controller
- Brightness. The brightness level of the emulator is always High.
- High-resolution video. Videos with a resolution higher than VGA resolution (640 x 480) cannot be displayed reliably, especially on emulator images with only 512MB of memory.

Mouse input

Simulate mouse input using the physical mouse or trackpad on your Windows PC and the mouse input button on the emulator toolbar. This feature is useful if your app

provides the user with an ability to utilize a mouse paired to their Windows 10 device to provide input.

Tap the mouse input button on the emulator toolbar to enable mouse input. Any click events within the emulator chrome will now be sent to the Windows 10 Mobile OS running inside the emulator VM as mouse events.



The emulator screen with the mouse input enabled.



The mouse input button on the emulator toolbar.

Keyboard input

The emulator supports mapping of the hardware keyboard on your development computer to the keyboard on a Windows Phone. The behavior of the keys is the same as on a Windows Phone device.

By default, the hardware keyboard is not enabled. This implementation is equivalent to a sliding keyboard that must be deployed before you can use it. Before you enable the hardware keyboard, the emulator accepts key input only from the control keys.

Special characters on the keyboard of a localized version of a Windows development computer are not supported by the emulator. To enter special characters that are present on a localized keyboard, use the Software Input Panel (SIP) instead.

To use your computer's keyboard in the emulator, press F4.

To stop using your computer's keyboard in the emulator, press F4.

The following table lists the keys on a hardware keyboard that you can use to emulate the buttons and other controls on a Windows Phone.

Note that in Emulator Build 10.0.14332 the computer hardware key mapping was changed. Values in the second column of the table below represent these new keys.

Computer hardware keys (Emulator Build 10.0.14295 and earlier)	Computer hardware keys (Emulator Build 10.0.14332 and newer)	Windows Phone hardware button	Notes
F1	WIN + ESC	BACK	Long presses work as expected.
F2	WIN + F2	START	Long presses work as expected.
F3	WIN + F3	SEARCH	
F4	F4 (no change)	Toggles between using the local computer's keyboard and not using the local computer's keyboard.	
F6	WIN + F6	CAMERA HALF	A dedicated camera button that is pressed halfway.
F7	WIN + F7	CAMERA FULL	A dedicated camera button.
F9	WIN + F9	VOLUME UP	
F10	WIN + F10	VOLUME DOWN	
F12	WIN + F12	POWER	Press the F12 key twice to enable the lock screen. Long presses work as expected.

Computer hardware keys (Emulator Build 10.0.14295 and earlier)	Computer hardware keys (Emulator Build 10.0.14332 and newer)	Windows Phone hardware button	Notes
ESC	WIN + ESC	BACK	Long presses work as expected.

Near Field Communications (NFC)

Build and test apps that use Near Field Communication (NFC) enabled features on Windows 10 Mobile by using the **NFC** tab of the emulator's **Additional Tools** menu. NFC is useful for a number of scenarios ranging from Proximity scenarios (such as tap to share) to card emulation (such as tap to pay).

You can test your app by simulating a pair of phones tapping together by using a pair of emulators, or you can test your app by simulating a tap to a tag. Also in Windows 10, mobile devices are enabled with HCE (Host Card Emulation) feature and by using the phone emulator you can simulate tapping your device to a payment terminal for APDU command-response traffic.

The NFC tab supports three modes:

- Proximity Mode
- HCE (Host Card Emulation) Mode
- Smart Card Reader Mode

In all modes, the emulator window has three areas of interest.

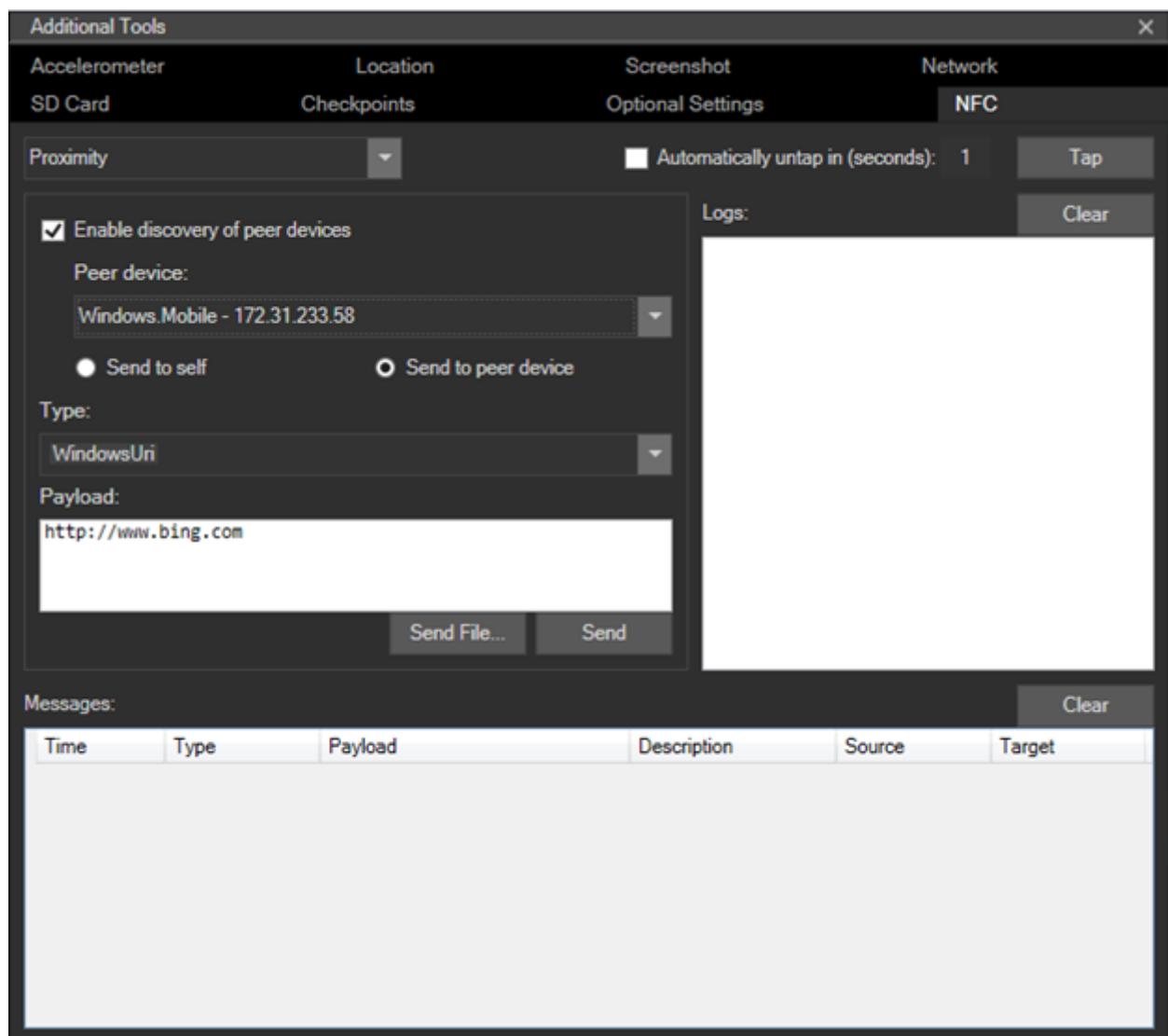
- The top left section is specific to the mode selected. The features of this section depend on the mode, and are detailed in the mode-specific sections below.
- The top right section lists the logs. When you tap a pair of devices together (or tap to the POS terminal) the tap event is logged and when the devices are untapped the untap event is logged. This section also records if your app responded before the connection is broken or any other action you have taken in the emulator UI with time stamps. Logs are persistent between mode switches, and you can clear the logs at any point by hitting the **Clear** button above the **Logs** screen.
- The bottom half of the screen is the message log and shows the transcript of all messages sent or received over the currently selected connection, depending on the mode selected.

Important When you first launch the tapper tool, you will get a Windows Firewall prompt. You **MUST** select ALL 3 check boxes and allow the tool through the firewall, or the tool will silently fail to work.

After launching the quick start installer, make sure you follow the above instruction to select all 3 check boxes on the firewall prompt. Also, the tapper tool must be installed and used on the same physical host machine as the Microsoft Emulator.

Proximity mode

To simulate a pair of phones tapping together you'll need to launch a pair of Windows Mobile emulators. Since Visual Studio doesn't support running two identical emulators at the same time, you'll need to select different resolutions for each of the emulators as a workaround.



When you check the **Enable discovery of peer devices** checkbox, the **Peer device** dropdown box shows Microsoft Emulators (running on the same physical host machine

or in the local network) as well as the Windows machines running the simulator driver (running on the same machine or in the local network).

Once both emulators are running:

- Select the emulator you would like to target in the **Peer device** list.
- Select the **Send to peer device** radio button.
- Click **Tap** button. This will simulate the two devices tapping together and you should be hearing the NFC tap notification sound
- To disconnect the 2 devices, simply hit the **Untap** button.

Alternatively, you can enable **Automatically untap in (seconds)** check box where you can specify the number of seconds you want the devices to be tapped and they will be automatically untapped after the specified number of seconds (simulating what would be expected of a user in real life, they would only hold their phones together for a short time). Note however that currently the message log isn't available after the connection has been untapped.

To simulate reading messages from a tag or receiving messages from another device:

- Select the **Send to self** radio button to test scenarios that require only one NFC enabled device.
- Click **Tap** button. This will simulate the tapping a device to a tag and you should be hearing the NFC tap notification sound
- To disconnect, simply hit the **Untap** button.

Using the proximity mode you can inject messages as if they came from a tag or another peer device. The tool allows you to send messages of the following types.

- WindowsURI
- WindowsMime
- WritableTag
- Pairing:Bluetooth
- NDEF
- NDEF:MIME
- NDEF:URI
- NDEF:wkt.U

You can either create these messages by editing the **Payload** windows or providing them in a file. For more information about these types and how to use them please refer to the Remarks section of the [ProximityDevice.PublishBinaryMessage](#) reference page.

The Windows Driver Kit (WDK) includes an [NFC CX client driver sample](#).

Host Card Emulation (HCE) Mode

In Host Card Emulation (HCE) mode you can test your HCE-based card emulation application by writing your own custom scripts to simulate a smart card reader terminal, such as a Point of Sale (POS) terminal. This tool assumes that you are familiar with the command response pairs (compliant with ISO-7816-4) that are sent between a reader terminal (such as POS, badge reader or transit card reader) and the smart card (that you are emulating in your application).

Additional Tools

Accelerometer

Location

Screenshot

Network

SD Card

Checkpoints

Optional Settings

NFC

Host Card Emulation (HCE)

☐ Automatically untap in (seconds): 1

Untap

Sample Script

```
public static void Run(IScriptProcessor processor)
{
    // POS sends Select PPSE command:
    processor.Send("00A404000E325041592E5359532E444446303100");
    // POS sends Select AID command:
    processor.Send("00A4040007A000000003101000");
    // POS sends Get Processing Options command:
    processor.Send("80A80000048302800000");
    // POS sends Read Record command, requesting the payment data
    processor.Send("00B2010C00");
}
```

☐ Enable MasterCard listener

Logs:

```
17:05:58.219: Devices are tapped together.
17:05:58.472: Launching script: Sample Script
17:06:02.486: Response wasn't received within
timeout. Resuming script.
17:06:06.492: Response wasn't received within
timeout. Resuming script.
17:06:10.368: Script stopped running.
```

Clear

Messages:

Time	Type	Payload	Description	Source	Target
17:05:58.473	HCECommand	00-A4-04-00-0E-325041592E5359532E444446303100	SELECT FILE		169.254.86.176
17:06:02.486	HCECommand	00-A4-04-00-07-A000000003101000	SELECT FILE		169.254.86.176
17:06:06.492	HCECommand	80-A8-00-00-04-83028000-00	GET PROCESSING OF		169.254.86.176

Clear

- Create a new script by clicking the **Add** button in the script editor section. You can provide a name for your script and after you are done with editing, you can save your script using the **Save** button.
- Your saved scripts will be available the next time you launch the emulator.
- Run your scripts by hitting the **Play** button in the scripts editor window. This action results in simulating of tapping your phone to the terminal and sending commands written in your script. Alternatively you can hit the **Tap** button and then the **Play** button, until you hit **Play** the script will not run.

- Stop sending commands by hitting the **Stop** button, which stops sending the commands to your application but the devices remain tapped until you hit the **Untap** button.
- Delete your scripts by selecting the script in the dropdown menu and hitting **Delete** button.
- The emulator tool does not check for the syntax of your scripts until you run the script using the **Play** button. The messages sent by your script are dependent on your implementation of your card emulation app.

To learn more about HCE support and how to develop HCE apps in Windows 10 Mobile, please refer to the [Microsoft NFC Team Blog](#).

How to Create Scripts for HCE Testing

The scripts are written as C# code and your script's Run method is called when you click the **Play** button, this method takes an IScriptProcessor interface which is used to transceive APDU commands, output to the log window, and control the timeout for waiting on an APDU response from the phone.

Below is a reference on what functionality is available:

C#

```
public interface IScriptProcessor
{
    // Sends an APDU command given as a hex-encoded string, and
    returns the APDU response
    string Send(string s);

    // Sends an APDU command given as a byte array, and returns the
    APDU response
    byte[] Send(byte[] s);

    // Logs a string to the log window
    void Log(string s);

    // Logs a byte array to the log window
    void Log(byte[] s);

    // Sets the amount of time the Send functions will wait for an
    APDU response, after which
    // the function will fail
    void SetResponseTimeout(double seconds);
}
```

Smart Card Reader Mode

The emulator can be connected to a smart card reader device on your host computer, such that smart cards inserted or tapped will show up to your phone application and can be communicated to with APDUs using the [Windows.Devices.SmartCards.SmartCardConnection](#) class. For this to work, you will need a compatible smart card reader device attached to your computer, USB smart card readers (both NFC/contactless and insert/contact) are widely available. To enable the emulator to work with an attached smart card reader, first choose the **Card Reader** mode which should show a dropdown box listing all the compatible smart card readers attached to the host system, then choose the smart card reader device you'd like to be connected from the dropdown.

Note that not all NFC-capable smart card readers support some types of NFC cards, and some do not support the standard PC/SC storage card APDU commands.

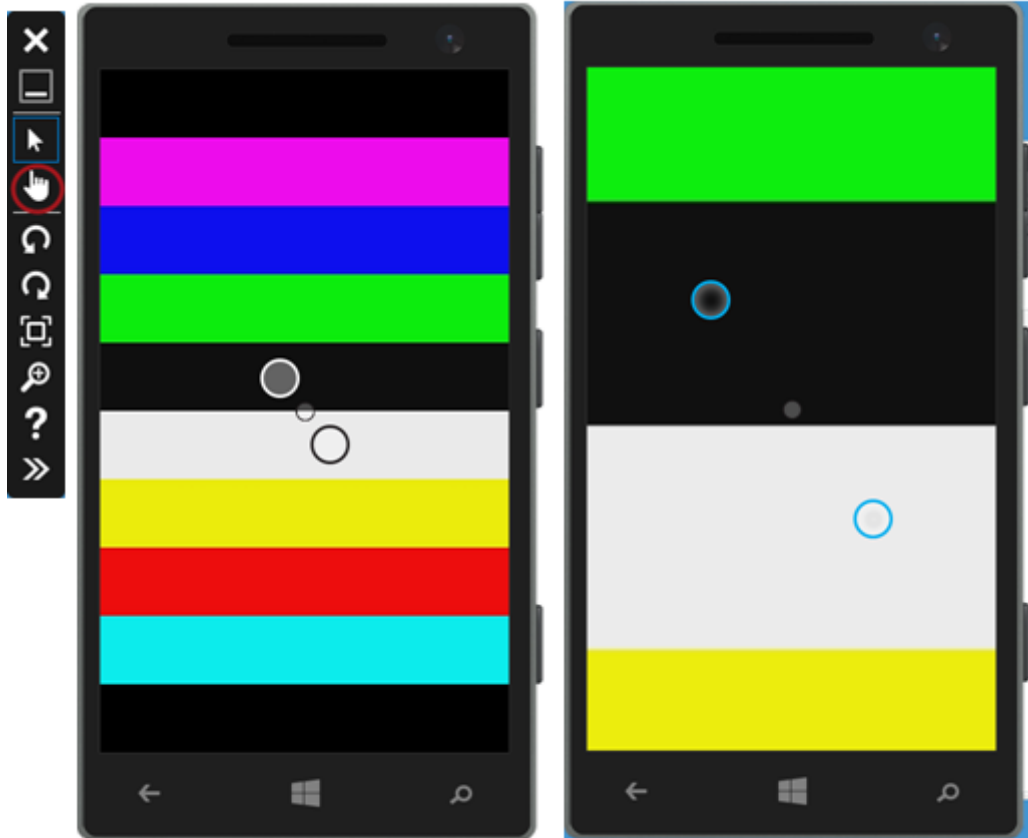
Multi-point input

Simulate multi-touch input for pinching and zooming, rotating, and panning objects by using the **Multi-touch Input** button on the emulator toolbar. This feature is useful if your app displays photos, maps, or other visual elements that users can pinch and zoom, rotate, or pan.

1. Tap the **Multi-touch Input** button on the emulator toolbar to enable multi-point input. Two touch points appear on the emulator screen around a center point.
2. Right-click and drag one of the touch points to position them without touching the screen.
3. Left-click and drag one of the touch points to simulate pinching and zooming, rotating, or panning.
4. Tap the **Single Point Input** button on the emulator toolbar to restore normal input.

The following screenshot shows multi-touch input.

1. The small left image shows the **Multi-touch Input** button on the emulator toolbar.
2. The middle image shows the emulator screen after tapping the **Multi-touch Input** button to display the touch points.
3. The right image shows the emulator screen after dragging the touch points to zoom the image.



Accelerometer

Test apps that track the movement of the phone by using the **Accelerometer** tab of the emulator's **Additional Tools**.

You can test the accelerometer sensor with live input or pre-recorded input. The only type of recorded data that's available simulates shaking the phone. You can't record or save your own simulations for the accelerometer.

1. Select the desired starting orientation in the **Orientation** drop-down list.
2.
 - Select the type of input.

To run the simulation with live input

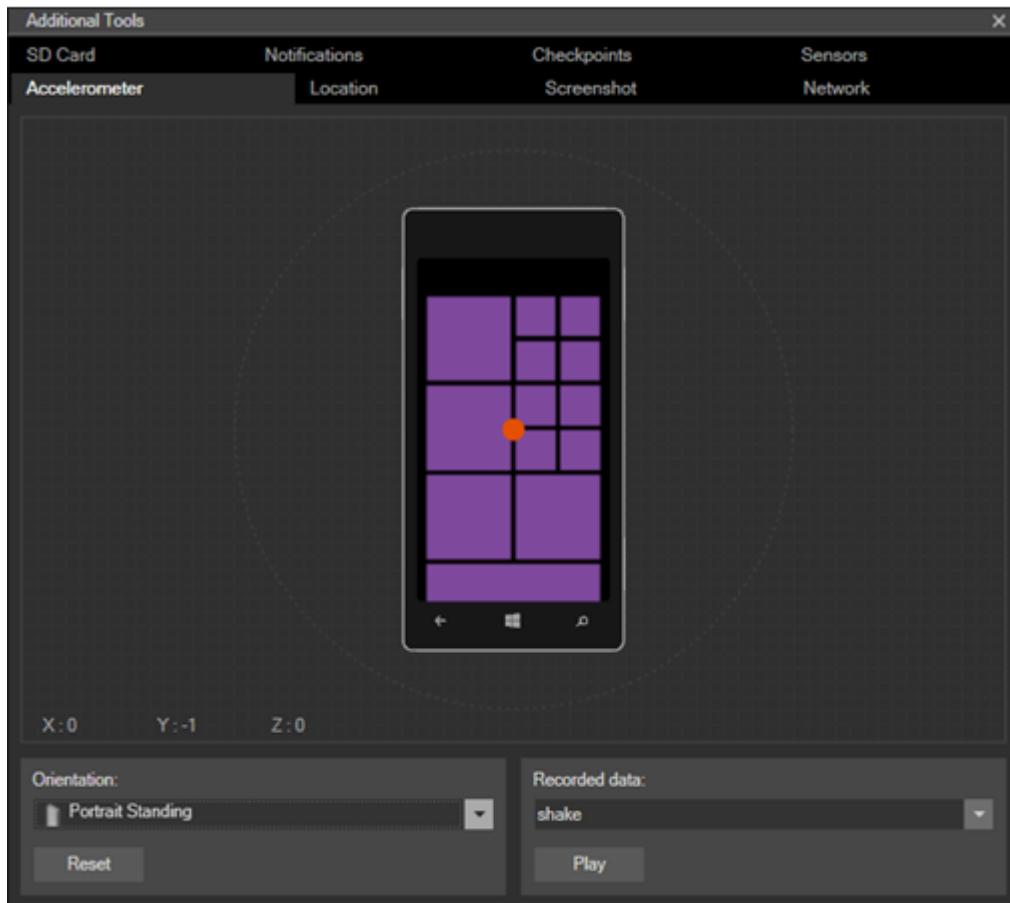
In the middle of the accelerometer simulator, drag the colored dot to simulate movement of the device in a 3D plane.

Moving the dot on the horizontal axis rotates the simulator from side to side. Moving the dot on the vertical axis rotates the simulator back and forth, rotating around the x-axis. As you drag the dot, the X, Y, and Z coordinates update based on the rotation calculations. You cannot move the dot outside the bounding circle in the touch pad area.

Optionally, click **Reset** to restore the starting orientation.

- **To run the simulation with recorded input**

In the **Recorded Data** section, click the **Play** button to start playback of the simulated data. The only option available in the **Recorded Data** list is shake. The simulator does not move on the screen when it plays back the data.



Location and driving

Test apps that use navigation or geofencing by using the **Location** tab of the emulator's **Additional Tools**. This feature is useful for simulating driving, biking, or walking in conditions similar to the real world.

You can test your app while you simulate moving from one location to another at different speeds and with different accuracy profiles. The location simulator can help you to identify changes in your usage of the location APIs usage that improve the user experience. For example, the tool can help you identify that you have to tune geofence parameters, such as size or dwell time, to detect the geofences successfully in different scenarios.

The **Location** tab supports three modes. In all modes, when the emulator receives a new position, that position is available to trigger the **PositionChanged** event or to respond to a **GetGeopositionAsync** call in your location-aware app.

- In **Pin** mode, you place pushpins on the map. When you click **Play all points**, the location simulator sends the location of each pin to the emulator one after another, at the interval specified in the **Seconds per pin** text box.
- In **Live** mode, you place pushpins on the map. The location simulator sends the location of each pin to the emulator immediately as you place them on the map.
- In **Route** mode, you place pushpins on the map to indicate waypoints, and the location simulator automatically calculates a route. The route includes invisible pins at one-second intervals along the route. For example, if you have select the **Walking** speed profile, which assumes a speed of 5 kilometers per hour, then invisible pins are generated at intervals of 1.39 meters. When you click **Play all points**, the location simulator sends the location of each pin to the emulator one after another, at the interval determined by the speed profile selected in the drop-down list.

In all modes of the location simulator, you can do the following things.

- You can search for a location by using the **Search** box.
- You can **Zoom in** and **Zoom out** on the map.
- You can save the current set of data points to an XML file, and reload the file later to reuse the same data points.
- You can **Toggle pushpin mode on or off** and **Clear all points**.

In Pin and Route mode, you can also do the following things.

- Save a route you created for later use.
- Load a route previously created. You can even load route files created in previous versions of the tool.
- Modify a route by deleting pushpins (in Pin mode) or waypoints (in Route mode).

Accuracy profiles

In all modes of the location simulator, you can select one of the following accuracy profiles in the **Accuracy profile** drop-down list.

Profile	Description
Pinpoint	Assumes perfectly accurate location readings. This setting is not realistic, but it's useful for testing the logic of your app.
Urban	Assumes that buildings are restricting the number of satellites in view, but there is often a high density of cell towers and Wi-Fi access points that can be used for positioning.
Suburban	Assumes that satellite positioning is relatively good and there is good density of cell towers, but the density of Wi-Fi access points is not high.
Rural	Assumes that satellite positioning is good, but there is low density of cell towers and almost no Wi-Fi access points that can be used for positioning.

Speed profiles

In **Route** mode, you can select one of the following speed profiles in the drop-down list.

Profile	Speed per hour	Speed per second	Description
Speed Limit	Speed limit of the route	Not applicable	Traverse the route at the posted speed limit.
Walking	5 km/h	1.39 m	Traverse the route at a natural walking pace of 5 km/h.
Biking	25 km/h	6.94 m	Traverse the route at a natural biking pace of 25 km/h.
Fast			Traverse the route faster than the posted speed limit.

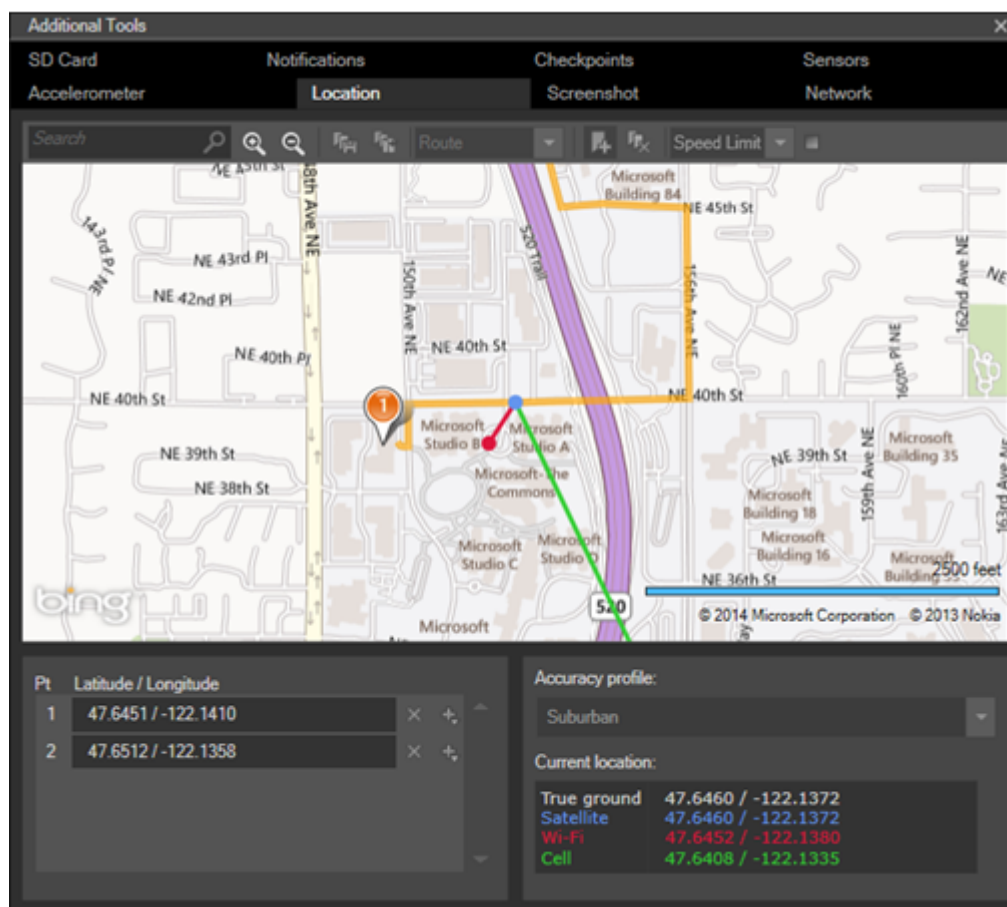
Route mode

Route mode has the following features and limitations.

- Route mode requires an Internet connection.
- When the Urban, Suburban, or Rural accuracy profile is selected, the location simulator calculates a simulated satellite-based position, a simulated Wi-Fi position, and a simulated cellular position for each pin. Your app receives only one of these positions. The three sets of coordinates for the current location are displayed in different colors on the map and in the **Current location** list.
- The accuracy of the pins along route the route is not uniform. Some of the pins use satellite accuracy, some use Wi-Fi accuracy, and some use cellular accuracy.

- You cannot select more than 20 waypoints for the route.
- Positions for the visible and invisible pins on the map are generated only once when you select a new accuracy profile. When you play the route more than once with the same accuracy profile during the same emulator session, the previously generated positions are reused.

The following screenshot shows Route mode. The orange line indicates the route. The blue dot indicates the accurate location of the car determined by satellite-based positioning. The red and green dots indicate less accurate locations calculated by using Wi-Fi and cellular positioning and the Suburban accuracy profile. The three calculated locations are also displayed in the **Current location** list.



More info about the location simulator

- You can request a position with the accuracy set to Default.
- When you test geofencing in the emulator, create a simulation that gives the geofencing engine a "warm-up" period to learn and adjust to the movement patterns.
- The only position properties that are simulated are the Latitude, Longitude, Accuracy, and PositionSource. The location simulator does not simulate other

properties such as Speed, Heading, and so forth.

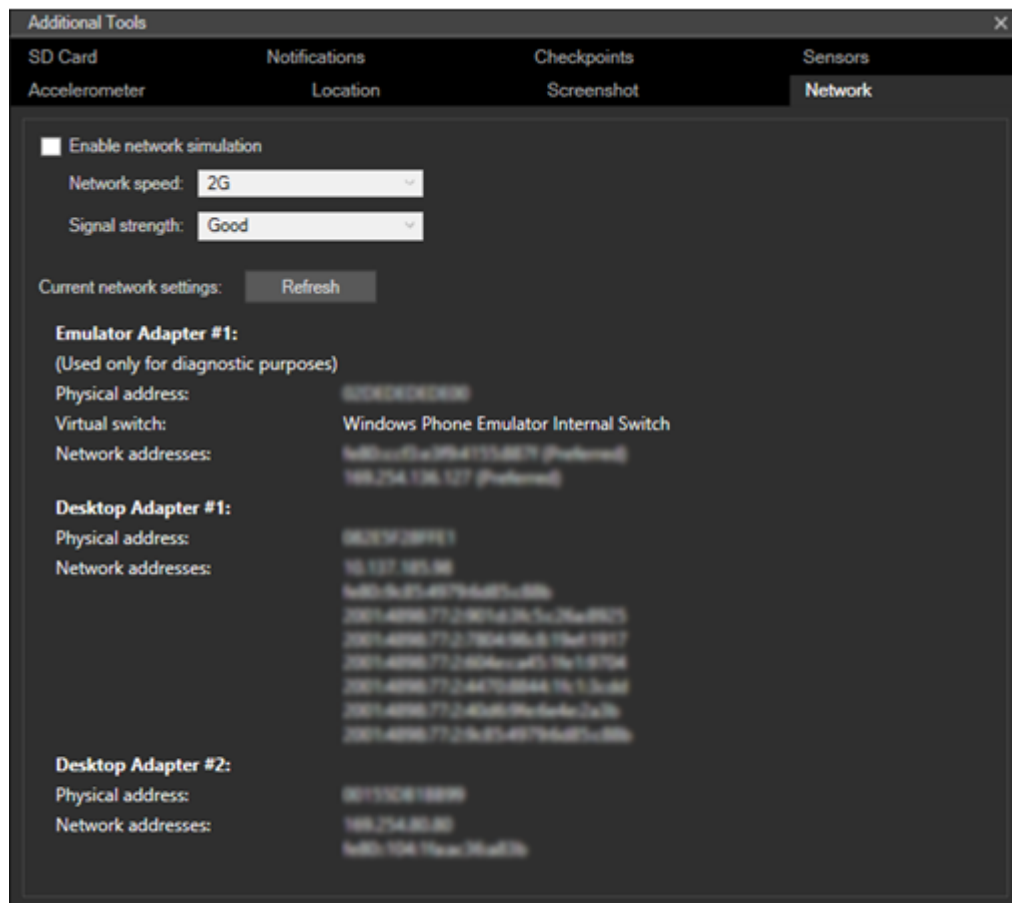
Network

Test your app with different network speeds and different signal strengths by using the **Network** tab of the emulator's **Additional Tools**. This feature is useful if your app calls web services or transfers data.

The network simulation feature helps you to make sure that your app runs well in the real world. The Windows Phone Emulator runs on a computer that usually has a fast WiFi or Ethernet connection. Your app, however, runs on phones that are typically connected over a slower cellular connection.

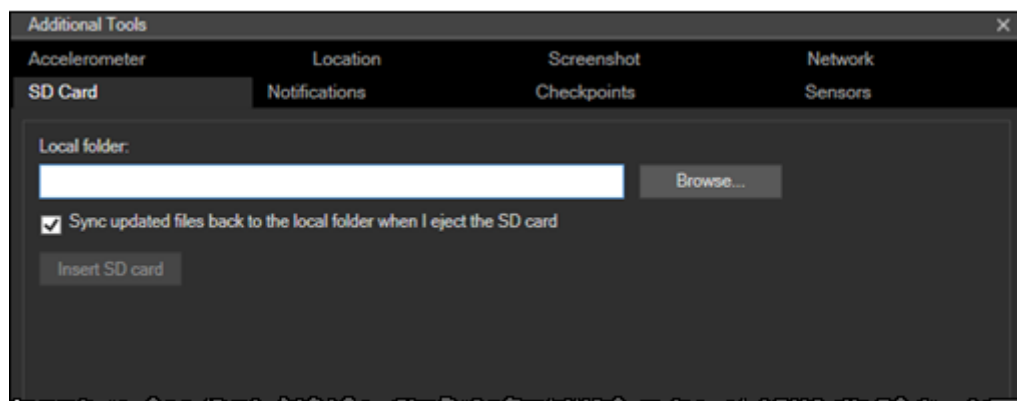
1. Check **Enable network simulation** to test your app with different network speeds and different signal strengths.
2. In the **Network speed** dropdown list, select one of the following options:
 - No network
 - 2G
 - 3G
 - 4G
3. In the **Signal strength** dropdown list, select one of the following options:
 - Good
 - Average
 - Poor
4. Clear **Enable network simulation** to restore the default behavior, which uses the network settings of your development computer.

You can also review the current network settings on the **Network** tab.



SD card

Test your app with a simulated removable SD card by using the **SD Card** tab of the emulator's **Additional Tools**. This feature is useful if your app reads or write files.



The **SD Card** tab uses a folder on the development computer to simulate a removable SD card in the phone.

1. Select a folder.

Click **Browse** to pick a folder on the development computer to hold the contents of the simulated SD card.

2. Insert the SD card.

After selecting a folder, click **Insert SD card**. When you insert the SD card, the following things happen:

- If you didn't specify a folder, or the folder's not valid, an error occurs.
- The files in the specified folder on the development computer are copied to the root folder of the simulated SD card on the emulator. A progress bar indicates the progress of the sync operation.
- The **Insert the SD card** button changes to **Eject SD card**.
- If you click **Eject SD card** while the sync operation is in progress, the operation is canceled.

3. Optionally, select or clear **Sync updated files back to the local folder when I eject the SD card**.

This option is enabled by default. When this option is enabled, files are synced from the emulator back to the folder on the development computer when you eject the SD card.

4. Eject the SD card.

Click **Eject SD card**. When you eject the SD card, the following things happen:

- if you have selected **Sync updated files back to the local folder when I eject the SD card**, the following things happen:
 - The files on the simulated SD card on the emulator are copied to the specified folder on the development computer. A progress bar indicates the progress of the sync operation.
 - The **Eject SD card** button changes to **Cancel sync**.
 - If you click **Cancel sync** while the sync operation is in progress, the card is ejected and the results of the sync operation are incomplete.
- The **Eject SD card** button changes back to **Insert SD card**.

Note Since an SD card used by the phone is formatted with the FAT32 file system, 32GB is the maximum size.

The speed of reading from and writing to the simulated SD card is throttled to imitate real-world speeds. Accessing an SD card is slower than accessing the computer's hard drive.

Notifications

Send push notifications to your app by using the **Notifications** tab of the emulator's **Additional Tools**. This feature is useful if your app receives push notifications.

You can easily test push notifications without creating the working cloud service that's required after you publish your app.

1. **Enable simulation.**

After you select **Enabled**, all apps deployed on the emulator use the simulation engine instead of the WNS or MPN service until you disable simulation.

2. **Select an app to receive notifications.**

The **AppId** list is automatically populated with all apps deployed to the emulator that are enabled for push notifications. Select an app in the drop-down list.

If you deploy another push-enabled app after enabling simulation, click **Refresh** to add the app to the list.

3. **Select a notification channel.**

After you select an app in the **AppId** list, the **URI** list is automatically populated with all the notification channels registered for the selected app. Select a notification channel in the drop-down list.

4. **Select a notification type.**

After you select a notification channel in the **URI** list, the **Notification Type** list is automatically populated with all the types available for the notification service. Select a notification type in the drop-down list.

The simulator uses the Uri format of the notification channel to determine whether the app is using WNS or MPN push notifications.

Simulation supports all notification types. The default notification type is **Tile**.

- The following WNS notification types are supported.
 - Raw
 - Toast

When your app uses WNS notifications and you select the **Toast** notification type, the simulation tab displays the **Tag** and **Group** fields. You can select these options and enter **Tag** and **Group** values to manage toast notifications in the Notification Center.

- Tile
- Badge
- The following MPN notification types are supported.
 - Raw
 - Toast
 - Tile

5. Select a notification template.

After you select a notification type in the **Notification Type** list, the **Templates** list is automatically populated with all the templates available for the notification type. Select a template in the drop-down list.

Simulation supports all template types.

6. Optionally, change the notification payload.

After you select a template in the **Templates** list, the **Notification Payload** text box is automatically populated with a sample payload for the template. Review the sample payload in the **Notification Payload** text box.

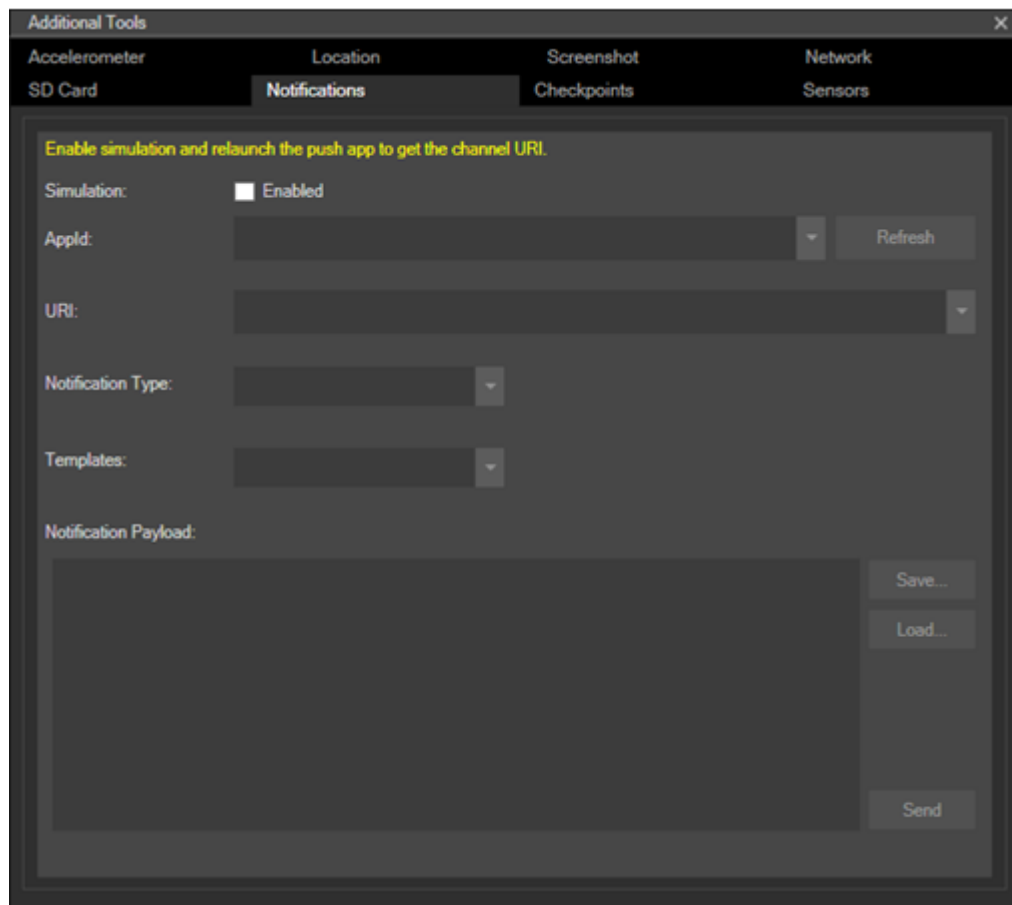
- You can send the sample payload without changing it.
- You can edit the sample payload in the text box.
- You can click **Load** to load a payload from a text or XML file.
- You can click **Save** to save the XML text of the payload to use again later.

The simulator does not validate the XML text of the payload.

7. Send the push notification.

Click **Send** to deliver the push notification to the selected app.

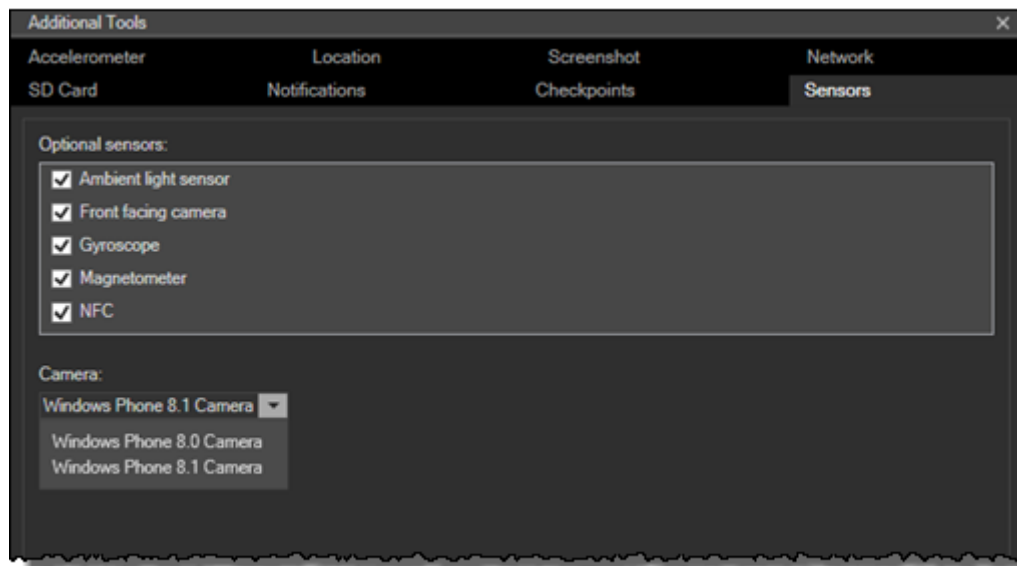
The screen displays a message to indicate success or failure.



Sensors

Test how your app works on low-cost phones that don't have all the optional sensors or camera features by using the **Sensors** tab of the emulator's **Additional Tools**. This feature is useful if your app uses the camera or some of the phone's sensors, and you want your app to reach the largest possible market.

- By default, all sensors are enabled in the **Optional sensors** list. Select or clear individual check boxes to enable or disable individual sensors.
- After you change your selections, click **Apply**. Then you have to restart the emulator.
- If you make changes, and then you switch tabs or close the **Additional Tools** window without clicking **Apply**, your changes are discarded.
- Your settings are persisted between for the emulator session until you change them or reset them. If you capture a checkpoint, the settings are saved with the checkpoint. The settings are persisted only for the specific emulator that you're using.



Sensor options

You can enable or disable the following optional hardware sensors:

- Ambient light sensor
- Front-facing camera
- Gyroscope
- Compass (magnetometer)
- NFC
- Software buttons (only on some high-resolution emulator images)

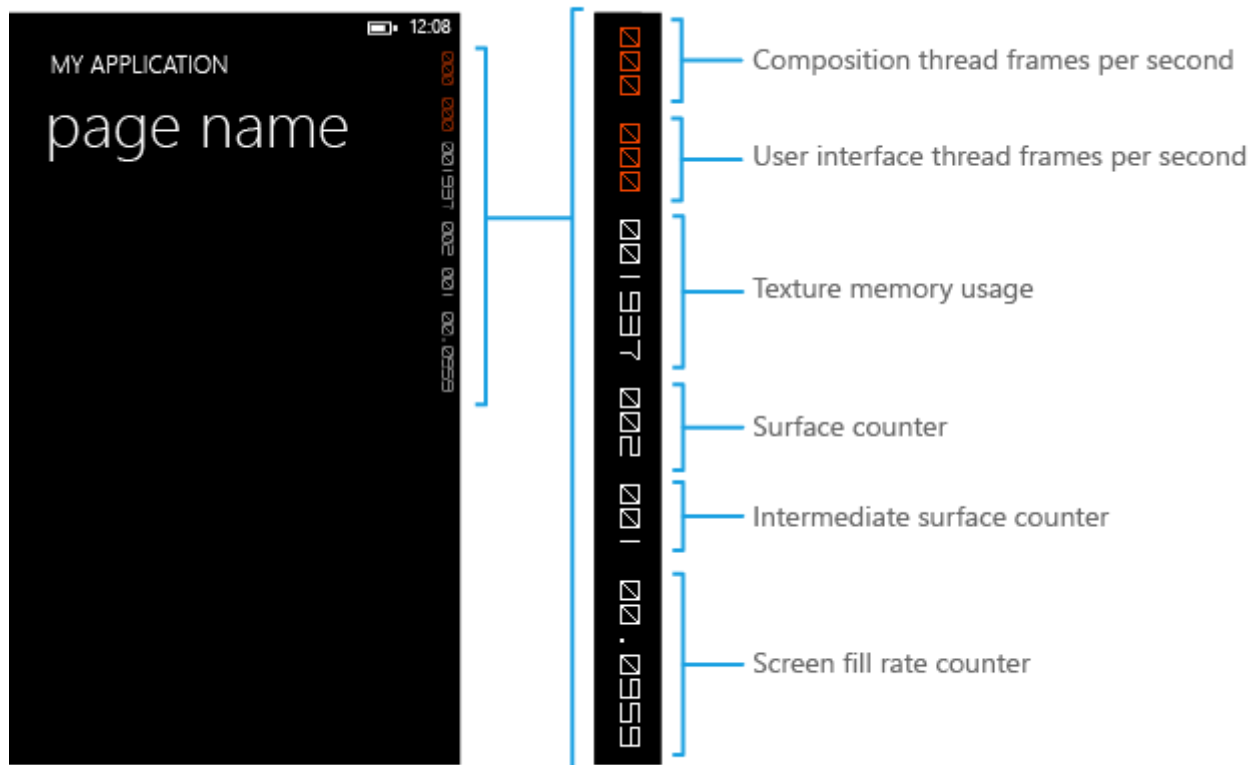
Camera options

You can enable or disable the optional front-facing camera by selecting or clearing the check box in the **Optional sensors** list.

You can also select a camera profile in the **Camera** dropdown list.

Frame rate counters

Use the frame rate counters in Windows Phone emulator to monitor the performance of your running app.



Descriptions of the frame rate counters

The following table describes each frame rate counter.

Frame rate counter	Description
Composition (Render) Thread Frame Rate (FPS)	The rate at which the screen is updated.
User Interface Thread Frame Rate (FPS)	The rate at which the UI thread is running.
Texture Memory Usage	The video memory and system memory copies of textures being used in the app.
Surface Counter	The number of explicit surfaces being passed to the GPU for processing.
Intermediate Surface Counter	The number of implicit surfaces generated as a result of cached surfaces.
Screen Fill Rate Counter	The number of pixels being painted per frame in terms of screens. A value of 1 represents the number of pixels in the current screen resolution – for example, 480 x 800 pixels.

Enabling and disabling the frame rate counters

You can enable or disable the display of the frame rate counters in your code. When you create a Windows Phone app project in Visual Studio, the following code to enable the frame rate counters is added by default in the file App.xaml.cs. To disable the frame rate counters, set **EnableFrameRateCounter** to **false** or comment out the line of code.

C#

```
// Show graphics profiling information while debugging.
if (System.Diagnostics.Debugger.IsAttached)
{
    // Display the current frame rate counters.
    Application.Current.Host.Settings.EnableFrameRateCounter = true;

    // other code...
}
```

VB

```
' Show graphics profiling information while debugging.
If System.Diagnostics.Debugger.IsAttached Then

    ' Display the current frame rate counters.
    Application.Current.Host.Settings.EnableFrameRateCounter = True

    ' other code...
End If
```

Known Issues

The following are known issues with the emulator, with suggested ways to work around problems if you encounter them.

Error message: “Failed while removing virtual Ethernet switch”

In certain situations, including after you update to a new Windows 10 flight, a virtual network switch associated with the emulator can get into a state where it can't be deleted through the user interface.

To recover from this situation run "netcfg -d" from an administrator command prompt: `C:\Program Files (x86)\Microsoft XDE\<version>\XdeCleanup.exe`. When the command is finished running, reboot your computer to complete the recovery process.

Note This command will delete all networking devices, not just those associated with the emulator. When your computer starts again, all hardware networking devices will be discovered automatically.

Unable to launch the emulators

Microsoft Emulator includes XDECleanup.exe, a tool that deletes all VMs, diff disks, and emulator specific network switches, and it ships with the emulator (XDE) binaries already. You should use this tool to clean up emulator VMs if they get into a bad state. Run the tool from an administrator command prompt: `C:\Program Files (x86)\Microsoft XDE\<version>\XdeCleanup.exe`

Note XDECleanup.exe deletes all emulator specific Hyper-V VMs, and it also deletes any VM checkpoints or saved states.

Uninstall Windows 10 for Mobile Image

When you install the emulator, a Windows 10 for Mobile VHD image is installed, which gets its own entry in the **Programs and Features** list in the Control Panel. If you wish to uninstall the image, find **Windows 10 for Mobile Image - <version>** in the list of installed programs, right-click on it, and choose **Uninstall**.

In the current release, you must then manually delete the VHD file for the emulator. If you installed the emulator to the default path, the VHD file is at `C:\Program Files (x86)\Windows Kits\10\Emulation\Mobile\<version>\flash.vhd`.

How to disable hardware accelerated graphics

By default, Windows 10 Mobile Emulator uses hardware accelerated graphics. If you are having trouble launching the emulator with hardware acceleration enabled, you can turn it off by setting a registry value.

To disable hardware acceleration:

1. Start Registry Editor.
2. Create the following registry subkey if it doesn't exist:
`HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Microsoft\Xde\10.0`
3. Right click the 10.0 folder, point to **New**, and then click **DWORD Value**.
4. Type **DisableRemoteFx**, and then press Enter.
5. Double-click **DisableRemoteFx**, enter 1 in the **Value** data box, select the **Decimal** option, and then click **OK**.

6. Close Registry Editor.

Note: After setting this registry value, you must delete the virtual machine in Hyper-V manager for the configuration that you launched in Visual Studio, and then relaunch the emulator with software-rendered graphics.

Support Resources

To find answers and solve problems as you start working with the Windows 10 tools, please visit [Windows 10 Tools forum](#) [↗]. To see all the forums for Windows 10 development, visit [this link](#) [↗].

Related topics

- [Run Windows Phone apps in the emulator](#)
- [Windows and Windows Phone SDK archive](#)

Test Surface Hub apps using Visual Studio

Article • 10/20/2022

The Visual Studio simulator provides an environment where you can design, develop, debug, and test Universal Windows Platform (UWP) apps, including apps that you have built for Microsoft Surface Hub. The simulator does not use the same user interface as Surface Hub, but it is useful for testing how your app looks and behaves with the Surface Hub's screen size and resolution.

For more information on the simulator tool in general, see [Run UWP apps in the simulator](#).

Add Surface Hub resolutions to the simulator

To add Surface Hub resolutions to the simulator:

1. Create a configuration for the 55" Surface Hub by saving the following XML code into a file named *HardwareConfigurations-SurfaceHub55.xml*.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<ArrayOfHardwareConfiguration
xmlns:xsd="http://www.w3.org/2001/XMLSchema"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <HardwareConfiguration>
    <Name>SurfaceHub55</Name>
    <DisplayName>Surface Hub 55"</DisplayName>
    <Resolution>
      <Height>1080</Height>
      <Width>1920</Width>
    </Resolution>
    <DeviceSize>55</DeviceSize>
    <DeviceScaleFactor>100</DeviceScaleFactor>
  </HardwareConfiguration>
</ArrayOfHardwareConfiguration>
```

2. Create a configuration for the 84" Surface Hub by saving the following XML code into a file named *HardwareConfigurations-SurfaceHub84.xml*.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<ArrayOfHardwareConfiguration
xmlns:xsd="http://www.w3.org/2001/XMLSchema"

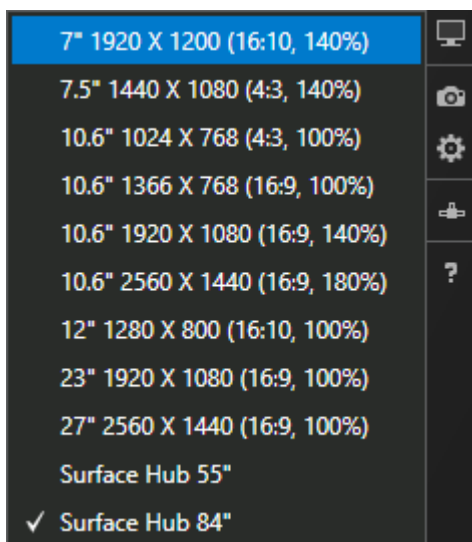
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <HardwareConfiguration>
    <Name>SurfaceHub84</Name>
    <DisplayName>Surface Hub 84"</DisplayName>
    <Resolution>
      <Height>2160</Height>
      <Width>3840</Width>
    </Resolution>
    <DeviceSize>84</DeviceSize>
    <DeviceScaleFactor>150</DeviceScaleFactor>
  </HardwareConfiguration>
</ArrayOfHardwareConfiguration>
```

- Copy the two XML files into *C:\Program Files (x86)\Common Files\Microsoft Shared\Windows Simulator\<version number>\HardwareConfigurations*.

ⓘ Note

Administrative privileges are required to save files into this folder.

- Run your app in the Visual Studio simulator. Click the **Change Resolution** button on the palette and select a Surface Hub configuration from the list.



💡 Tip

Turn on **Tablet mode (Windows 10 only)** to better simulate the experience of a Surface Hub.

Deploy apps to a Surface Hub device from Visual Studio

Manually deploying an app to a Surface Hub is a simple process.

Enable developer mode

By default, Surface Hub only installs apps from the Microsoft Store. To install apps signed by other sources, you must enable developer mode.

ⓘ Note

After developer mode has been enabled, you will need to reset the Surface Hub if you wish to disable it again. Resetting the device removes all local user files and configurations and then reinstalls Windows.

1. From the Surface Hub's **Start** menu, open the Settings app.

ⓘ Note

Administrative privileges are required to access the Settings app on Surface Hub.

2. Navigate to **Update & security > For developers**.
3. Choose **Developer mode** and accept the warning prompt.

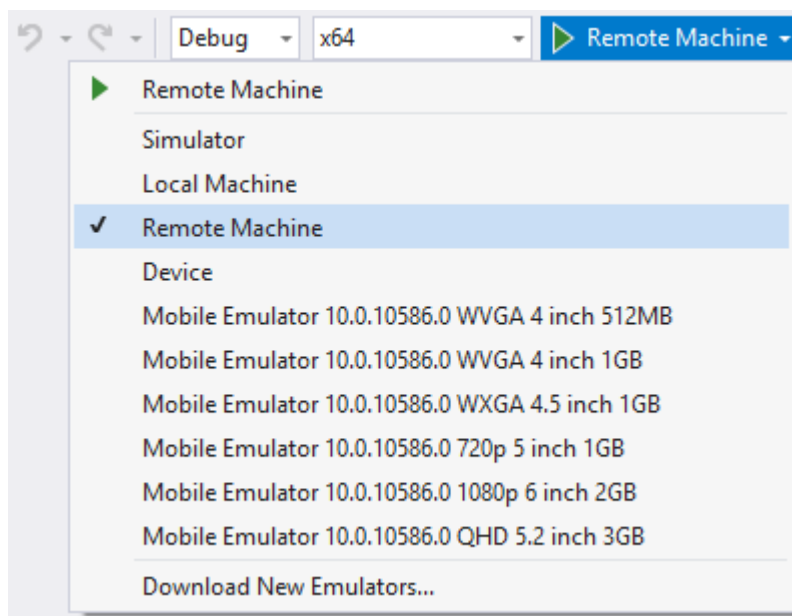
Deploy your app from Visual Studio

For more information on the deployment process in general, see [Deploying and debugging UWP apps](#).

ⓘ Note

This feature requires Visual Studio 2015 Update 1 or later, however we recommend that you use the latest most up to date version of Visual Studio. An up to date Visual Studio instance will give you all the latest development and security updates.

1. Navigate to the debug target dropdown next to the **Start Debugging** button and select **Remote Machine**.



2. Enter the Surface Hub's IP address. Ensure that the **Universal** authentication mode is selected.

 Tip

After you have enabled developer mode, you can find the Surface Hub's IP address on the welcome screen.

3. Select **Start Debugging (F5)** to deploy and debug your app on the Surface Hub, or press Ctrl+F5 to just deploy your app.

 Tip

If the Surface Hub is displaying the welcome screen, dismiss it by choosing any button.

Deploy an app through loose file registration

Article • 10/20/2022

This guide shows how to use the loose file layout to validate and share Windows 10 apps without needing to package them. Registering loose file layouts allows developers to quickly validate their apps without the need to package and install the apps.

What is a loose file layout?

Loose file layout is simply the act of placing app contents in a folder instead of going through the packaging process. The package contents are "loosely" available in a folder and not packaged.

Warning

Loose file layout registration is for developers and designers to quickly validate their apps during active development. This approach shouldn't be used to "dogfood" or flight the app. We recommend that the final validation be performed on a packaged app that is signed with a trusted certificate.

Advantages of loose file registration

- **Quick validation** - Because the app files are already unpacked, users can quickly register the loose file layout and launch the app. Just like a regular app, the user will be able to use the app as it was designed.
- **Easy in-network distribution** - If the loose files are located in a network share instead of a local drive, developers can send the network share location to other users who have access to the network, and they can register the loose file layout and run the app. This allows for multiple users to validate the app concurrently.
- **Collaboration** - Loose file registration allows developers and designers to continue working on visual assets while the app is registered. Users will see these changes when they launch the app. Note that you can only change static assets in this manner. If you need to modify any code or dynamically created content, you must re-compile the app.

How to register a loose file layout

Windows provides multiple developer tools to register loose file layouts on local and remote devices. You can choose from `WinAppDeployCmd` (Windows SDK Tool), Windows Device Portal, PowerShell, and [Visual Studio](#). Below we will go over how to register loose files using these tools. But first, ensure that you have following setup:

- Your devices must be on the Windows 10 Creators Update (Build 14965) or later.
- You will need to enable [developer mode](#) and [device discovery](#) on all devices.

Important

Loose file registration is only available on devices that support the Network Share (SMB) Protocol: Desktop and Xbox.

Register with WinAppDeployCmd

If you are using the SDK tools corresponding to the Windows 10 Creators Update (Build 14965) or later, you can use the `WinAppDeployCmd` command in a Command Prompt.

Windows Command Prompt

```
WinAppDeployCmd.exe registerfiles -remotedeploydir <Network Path> -ip <IP Address> -pin <target machine PIN>
```

Network Path – the path to the app's loose files.

IP Address – the IP Address of the target machine.

target machine PIN – A PIN, if required, to establish a connection with the target device. You will be prompted to retry with the `-pin` option if authentication is required. See [Device Discovery](#) to learn how to get a PIN.

Windows Device Portal

Windows Device Portal is available on all Windows 10 devices and is used by developers to test and validate their work. It caters to all audiences of the developer community with its browser UX and REST endpoints. For more information on Device Portal, see the [Windows Device Portal overview](#).

To register the loose file layout in Device Portal, follow these steps.

1. Connect to Device Portal by following the steps in the **Setup** section of the [Windows Device Portal overview](#).

2. In the Apps Manager tab, select **Register from Network Share**.
3. Enter the network share path to the loose file layout.
4. If the host device doesn't have access to the network share, there will be a prompt to enter the required credentials.
5. Once the registration is complete, you can launch the app.

On the Device Portal's Apps Manager page, you can also register optional loose file layouts for your main app by selecting the **I want to specify optional packages** checkbox and then specifying the network share paths of the optional apps.

PowerShell

Windows PowerShell also enables you to register loose file layouts, but only on the local device. If you need to register a layout to a remote device, you will need to use one of the other methods.

To register the loose file layout, launch PowerShell and enter the following.

PowerShell

```
Add-AppxPackage -Register <path to manifest file>
```

Troubleshooting

Mapped network drives

Currently, mapped network drives aren't supported for loose file registrations. Refer to the mapped drive with full the network share path.

Registration failure

The device on which the registration is taking place will need to have access to the file layout. If the file layout is hosted on a network share, ensure that the device has access.

Modifications to visual assets aren't being loaded in the app

The app will load its visual assets at launch time. If modifications were made to the visual assets after launching the app, you must re-launch the app to view the latest changes.

Beta testing

Article • 11/01/2022

Beta testing gives you the chance to improve your app based on feedback from individuals outside of your app-development team who try your unreleased app on their own devices.

This section describes your options for beta testing Universal Windows apps.

Why beta test?

To thoroughly test an app, you need to try it against as many device configurations and user interactions as possible. Doing all of that testing in-house is difficult if not impossible.

With beta testing, users try your app on their own devices. And it's unmoderated: instead of performing specified tasks, users have complete freedom in how they use an app, so they can find issues that you might never have expected.

With beta testing, you can:

- Test your app on a variety of devices.
- Identify performance issues and other bugs that you might not have found otherwise.
- Get real-world usage info that can be used improve the user experience.
- Receive feedback without affecting public ratings in the Microsoft Store.

When to beta test

It's best to conduct beta testing as the final stage of testing before you release your app. At that point, you have tested the app as thoroughly as you can yourself, and you've covered all explicit use cases. Beta testing is not a substitute for other testing methods. Since beta testing is unmoderated, participants may not catch all bugs in your code because every tester's experience is self-directed and it's unlikely that they'll explore all features of the app. But beta-testing feedback can give you a final wave of real-world feedback that reveals issues that you might never have expected before you go live.

Next steps

In [Partner Center](#), you can limit distribution of your apps to only your testers, regardless of which operating systems your app targets. There's no need to create a separate version of your app with a separate name and package identity; you can do your testing, then create a new submission when you're ready to make the app available to everyone. (Of course, you can create a separate app for testing only if you prefer. If you do, make sure to give it a different name from what you intend as the final, public app name.)

See [Beta testing and targeted distribution](#) to learn how to submit your app to the Store for beta testing.

Windows Device Portal overview

Article • 06/02/2023

The Windows Device Portal (WDP) is a web server included with Windows devices that lets you configure and manage the settings for the device over a network or USB connection (local connections are also supported on devices with a web browser).

WDP also provides advanced diagnostic tools for troubleshooting and viewing the real-time performance of your Windows device.

WDP functionality is programmatically exposed through a collection of [REST APIs](#).

This article provides a general description of the Windows Device Portal and includes links to articles with more specific information on each Windows device family.

ⓘ Note

A device family identifies the APIs, system characteristics, and behaviors that you can expect across a class of devices.

Setup

Each device family provides a version of the WDP, but features and setup vary based on the requirements of a device.

These are the basic steps for all devices.

1. Enable Developer Mode and Device Portal on your device (Settings -> Privacy & security -> For developers).
2. Connect your device and PC through a local network or with USB.
3. Navigate to the Device Portal page in your browser. This table shows the ports and protocols used by each device family.

The following table includes device-specific details for the WDP.

ⓘ Note

Windows Mixed Reality runs on regular desktop, so it's the same portal as Desktop.

Device family	On by default?	HTTP	HTTPS	USB	Instructions
Desktop and IoT Enterprise	Enable inside Dev Mode	50080*	50043*	N/A	Device Portal for Desktop or IoT Enterprise device
Xbox	Enable inside Dev Mode	Disabled	11443	N/A	Device Portal for Xbox
HoloLens	Yes, in Dev Mode	80 (default)	443 (default)	<code>http://127.0.0.1:10080</code>	Device Portal for HoloLens
IoT Core	Yes, in Dev Mode	8080	Enable via regkey	N/A	Device Portal for IoT Core
Phone	Enable inside Dev Mode	80	443	<code>http://127.0.0.1:10080</code>	Device Portal for Mobile

* This is not always the case, as Device Portal on desktop claims ports in the ephemeral range (>50,000) to prevent collisions with existing port claims on the device. To learn more, see the [Registry-based configuration](#) section in [Windows Device Portal for Desktop](#).

Features

Toolbar and navigation

The toolbar at the top of the page provides access to commonly used features.

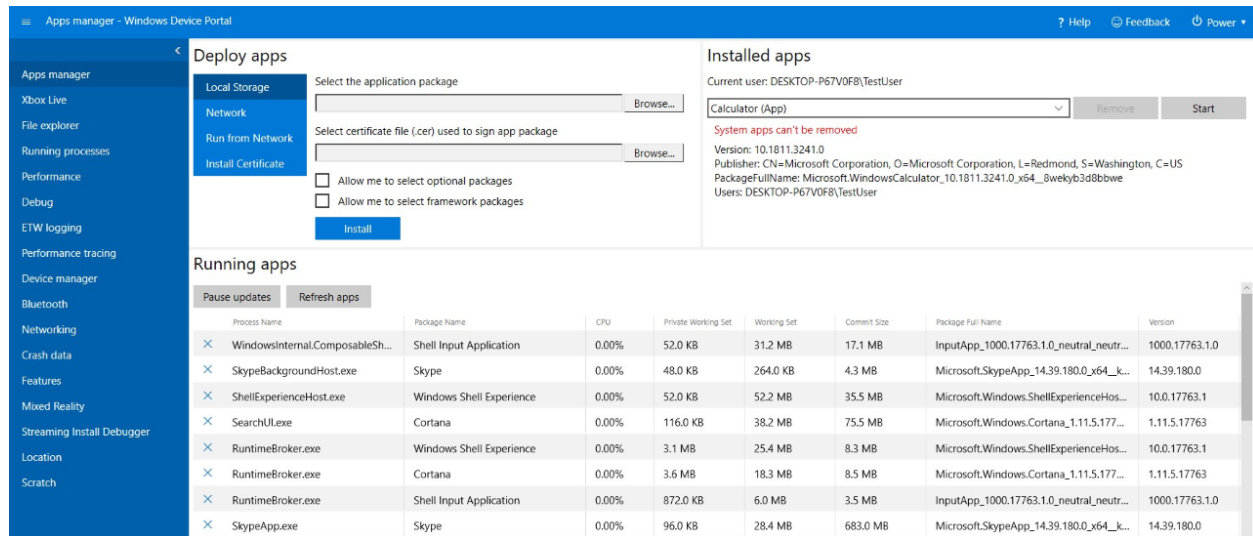
- **Power:** Access power options.
 - **Shutdown:** Turns off the device.
 - **Restart:** Cycles power on the device.
- **Help:** Opens the help page.

Use the links in the navigation pane along the left side of the page to navigate to the available management and monitoring tools for your device.

Tools that are common across device families are described here. Other options might be available depending on the device. For more info, see the specific page for your device type.

Apps manager

The Apps manager provides install/uninstall and management functionality for app packages and bundles on the host device.

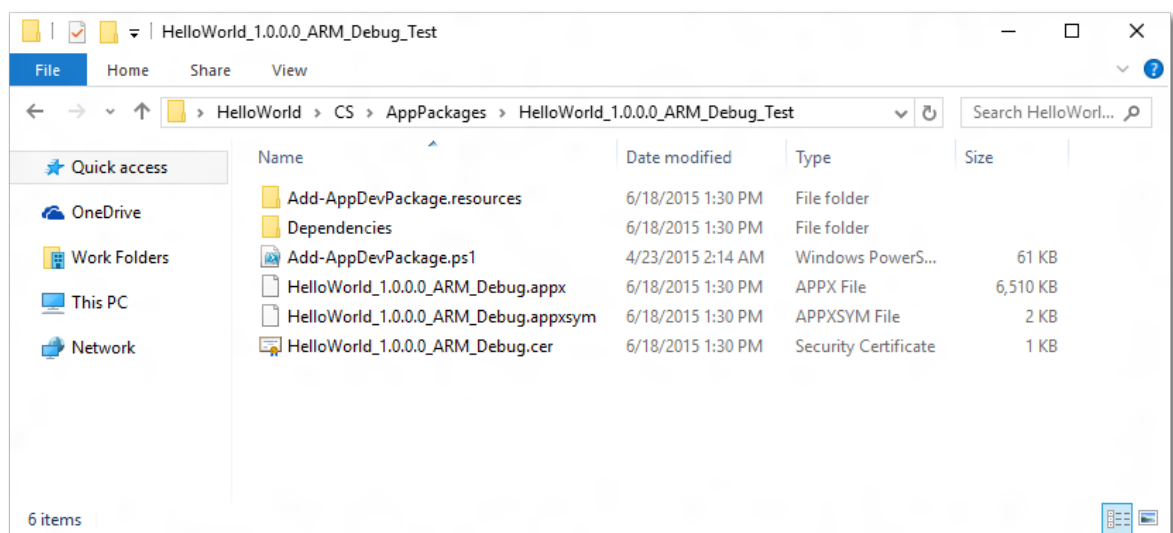


- **Deploy apps:** Deploy packaged apps from local, network, or web hosts and register loose files from network shares.
- **Installed apps:** Use the dropdown menu to remove or start apps that are installed on the device.
- **Running apps:** Get information about the apps that are currently running and close them as necessary.

Install (sideload) an app

You can sideload apps during development using Windows Device Portal:

1. When you've created an app package, you can remotely install it onto your device. After you build it in Visual Studio, an output folder is generated.



2. In Windows Device Portal, navigate to the **Apps manager** page.
3. In the **Deploy apps** section, select **Local Storage**.
4. Under **Select the application package**, select **Choose File** and browse to the app package that you want to sideload.
5. Under **Select certificate file (.cer) used to sign app package**, select **Choose File** and browse to the certificate associated with that app package.
6. Check the respective boxes if you want to install optional or framework packages along with the app installation, and select **Next** to choose them.
7. Select **Install** to initiate the installation.
8. If the device is running Windows 10 in S mode, and it is the first time that the given certificate has been installed on the device, restart the device.

Install a certificate

Alternatively, you can install the certificate via Windows Device Portal, and install the app through other means:

1. In Windows Device Portal, navigate to the **Apps manager** page.
2. In the **Deploy apps** section, select **Install Certificate**.
3. Under **Select certificate file (.cer) used to sign an app package**, select **Choose File** and browse to the certificate associated with the app package that you want to sideload.
4. Select **Install** to initiate the installation.
5. If the device is running Windows 10 in S mode, and it is the first time that the given certificate has been installed on the device, restart the device.

Uninstall an app

1. Ensure that your app is not running.
2. If it is, go to **Running apps** and close it. If you attempt to uninstall while the app is running, it will cause issues when you attempt to reinstall the app.
3. Select the app from the dropdown and click **Remove**.

Running processes

This page shows details about processes currently running on the host device. This includes both apps and system processes. On some platforms (Desktop, IoT, and HoloLens), you can terminate processes.

PID	Name	User name	Session Id	CPU	Private Working Set	Working Set	Commit Size
0	System Idle Process	NT AUTHORITY\SYSTEM	0	89.90%	8.0 KB	8.0 KB	N/A
4	System	NT AUTHORITY\SYSTEM	0	1.46%	4.0 KB	4.0 KB	N/A
436	smss.exe	NT AUTHORITY\SYSTEM	0	0.00%	44.0 KB	124.0 KB	N/A
656	MicrosoftEdge.exe	hirsinmob\DefApps	0	0.00%	5.9 MB	33.6 MB	20.3 MB
672	RuntimeBroker.exe	hirsinmob\DefApps	0	0.00%	10.0 MB	32.3 MB	19.2 MB
744	WUDFHost.exe	NT AUTHORITY\LOCAL...	0	0.00%	280.0 KB	884.0 KB	1.3 MB
812	csrss.exe	NT AUTHORITY\SYSTEM	0	0.00%	176.0 KB	456.0 KB	N/A
988	wininit.exe	NT AUTHORITY\SYSTEM	0	0.00%	120.0 KB	328.0 KB	N/A
1020	services.exe	NT AUTHORITY\SYSTEM	0	0.00%	1.4 MB	2.6 MB	N/A
1028	lsass.exe	NT AUTHORITY\SYSTEM	0	0.00%	3.0 MB	7.7 MB	N/A
1132	svchost.exe	NT AUTHORITY\SYSTEM	0	0.24%	4.1 MB	9.8 MB	7.7 MB
1196	svchost.exe	NT AUTHORITY\NETWO...	0	0.24%	3.2 MB	5.0 MB	4.6 MB
1260	WUDFHost.exe	NT AUTHORITY\LOCAL...	0	0.00%	276.0 KB	576.0 KB	1.4 MB

File explorer

This page allows you to view and manipulate files stored by any sideloaded apps. See the [Using the App File Explorer](#) blog post to learn more about the File explorer and how to use it.

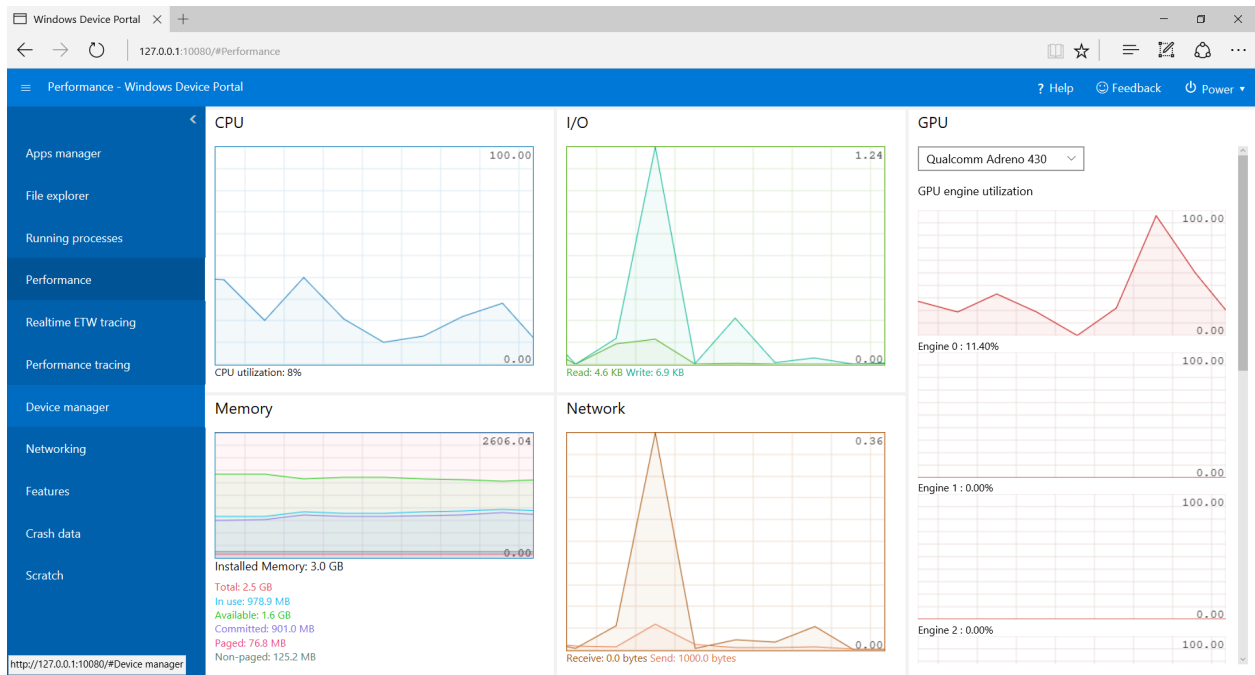
Type	Name	Date Created	File Size	Save	Delete	Rename
📁	My Rubber Duck (97ecfe2d-b105-4...					
📁	Sunrise IoT_WDP (caf03091-2975-...					

Performance

The Performance page shows real-time graphs of system diagnostic info like power usage, frame rate, and CPU load.

These are the available metrics:

- **CPU:** Percent of total available CPU utilization
- **Memory:** Total, in use, available, committed, paged, and non-paged
- **I/O:** Read and write data quantities
- **Network:** Received and sent data
- **GPU:** Percent of total available GPU engine utilization



Event Tracing for Windows (ETW) logging

The ETW logging page manages real-time Event Tracing for Windows (ETW) information on the device.

The screenshot shows the 'Realtime ETW tracing - Windows Device Portal' interface. The left sidebar is the same as the previous screenshot, with 'Realtime ETW tracing' selected. The main content area is titled 'ETW tracing' and includes the following sections:

- Hide Providers:** A checkbox that is currently unchecked.
- Registered providers:** A section with a dropdown menu showing 'Microsoft-WindowsPhone-GwpCPCProvider', a '5: Detailed trace' dropdown, and an 'Enable' button.
- Custom providers:** A section with a 'GUID (no brackets)' input field, a '5: Detailed trace' dropdown, and an 'Enable' button.
- Enabled providers:** A section with a dropdown menu, a 'Disable' button, and a 'Stop All' button.
- Providers history:** A section with a dropdown menu, a '5: Detailed trace' dropdown, an 'Enable' button, and a 'Clear' button.
- Filters:** A section with an 'ID' dropdown, an input field, and an 'Add' button.
- Events:** A section with a 'Clear' button and a 'Save to file' button.

At the bottom, there is a table header with columns: Timestamp (Local Time), Provider, ID, Task, Keyword, Level, and Payload.

Check **Hide providers** to show the Events list only.

- **Registered providers:** Select the event provider and the tracing level. The tracing level is one of these values:

1. Abnormal exit or termination
2. Severe errors
3. Warnings
4. Non-error warnings
5. Detailed trace

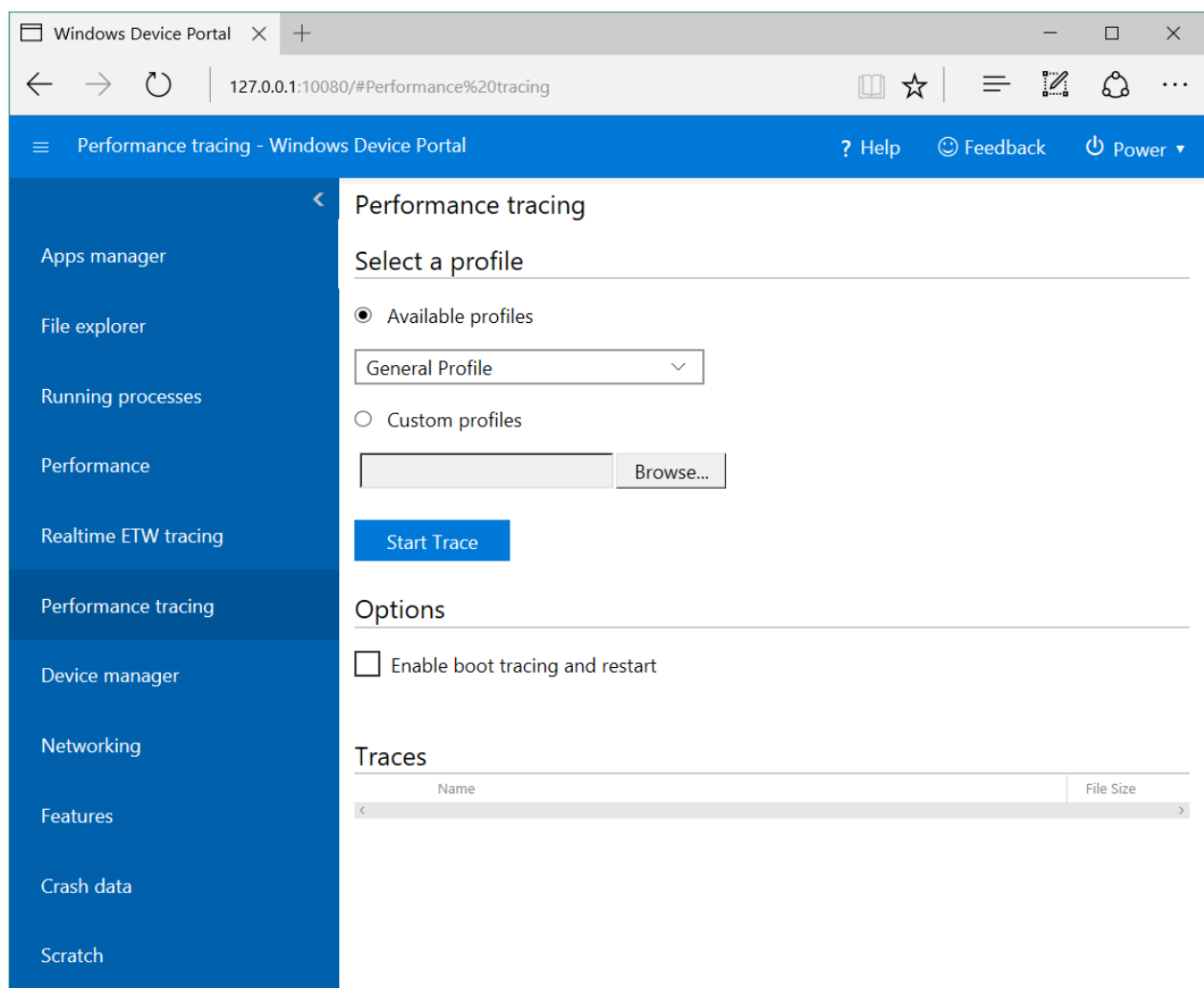
Click or tap **Enable** to start tracing. The provider is added to the **Enabled Providers** dropdown.

- **Custom providers:** Select a custom ETW provider and the tracing level. Identify the provider by its GUID. Do not include brackets in the GUID.
- **Enabled providers:** This lists the enabled providers. Select a provider from the dropdown and click or tap **Disable** to stop tracing. Click or tap **Stop all** to suspend all tracing.
- **Providers history:** This shows the ETW providers that were enabled during the current session. Click or tap **Enable** to activate a provider that was disabled. Click or tap **Clear** to clear the history.
- **Filters / Events:** The **Events** section lists ETW events from the selected providers in table format. The table is updated in real time. Use the **Filters** menu to set up custom filters for which events will be displayed. Click the **Clear** button to delete all ETW events from the table. This does not disable any providers. You can click **Save to file** to export the currently collected ETW events to a local CSV file.

For more details on using ETW logging, see the [Use Device Portal to view debug logs](#)  blog post.

Performance tracing

The [Windows Performance Toolkit](#) includes the Performance tracing page, which allows you to view the [Windows Performance Recorder \(WPR\)](#) traces from the host device.



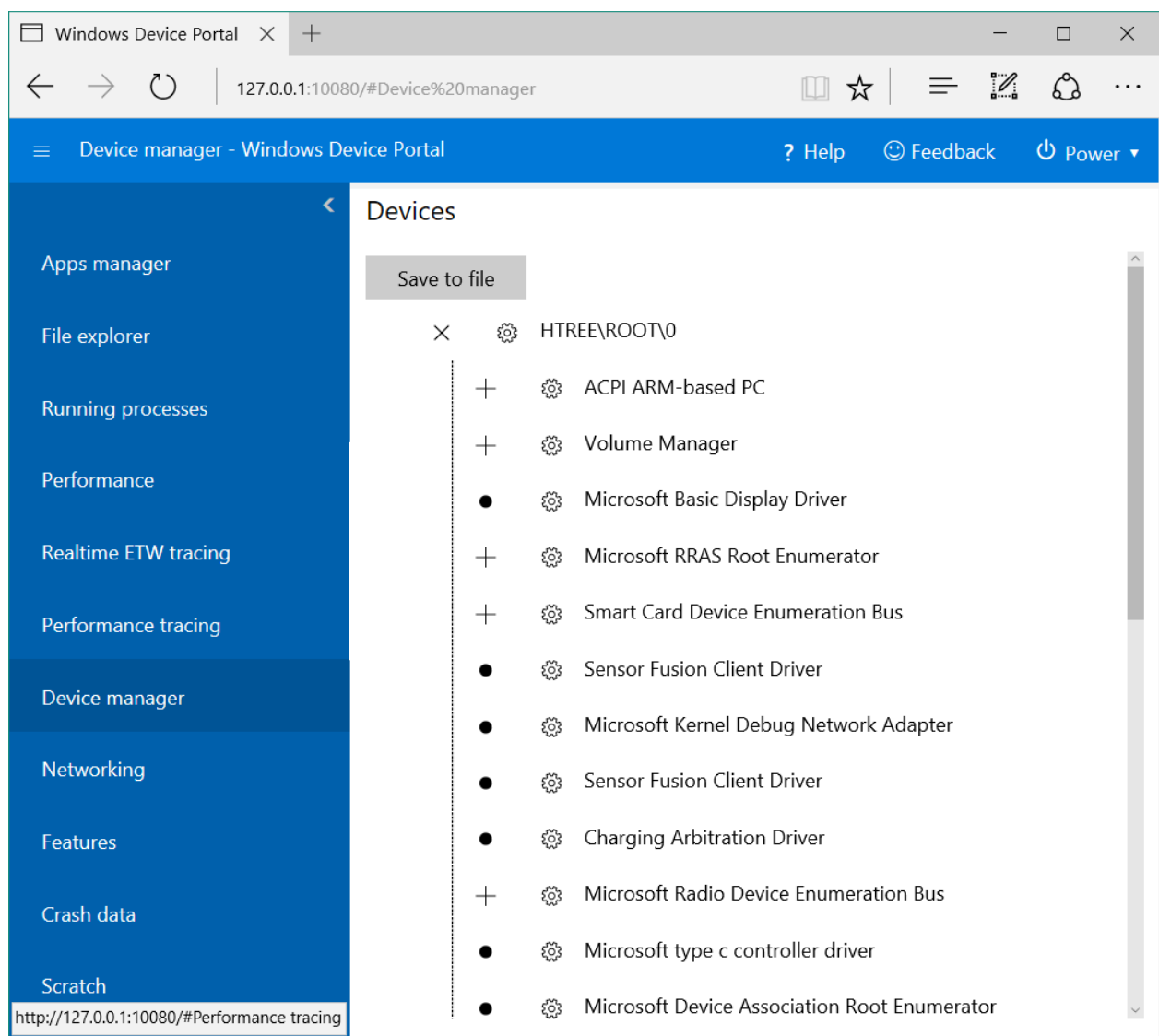
- **Available profiles:** Select the WPR profile from the dropdown, and click or tap **Start** to start tracing.
- **Custom profiles:** Click or tap **Browse** to choose a WPR profile from your PC. Click or tap **Upload and start** to start tracing.

To stop the trace, click **Stop**. Stay on this page until the trace file (.ETL) has finished downloading.

Captured .ETL files can be opened for analysis in the [Windows Performance Analyzer](#).

Device manager

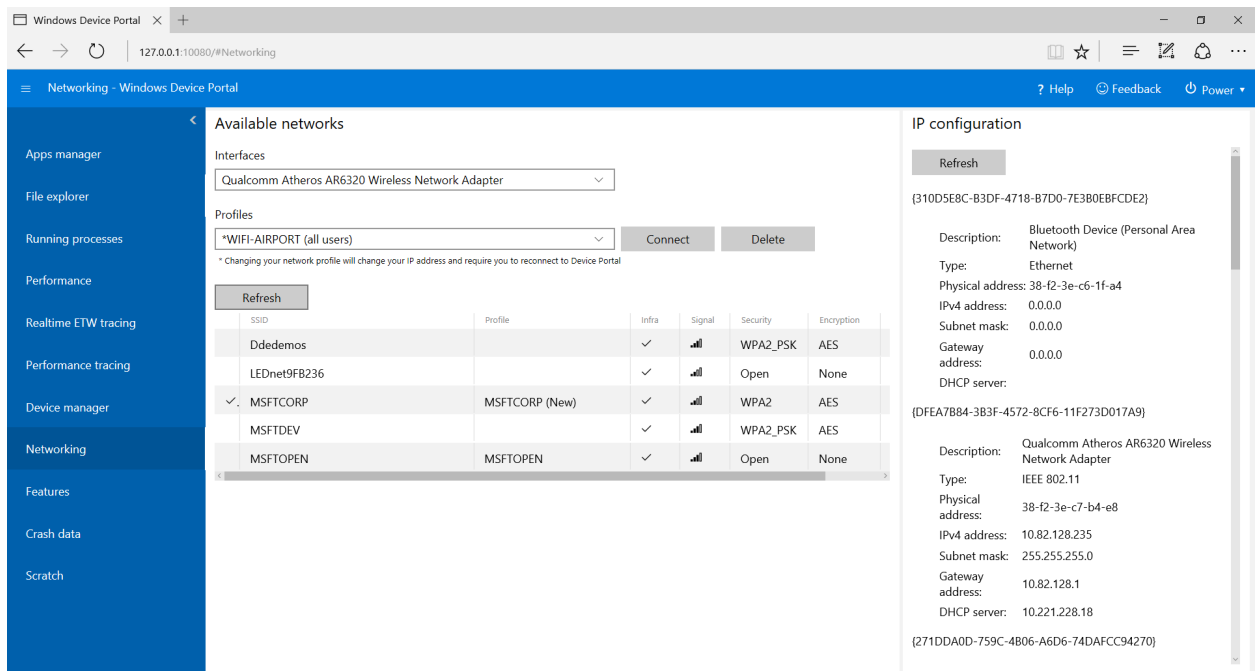
The Device manager page enumerates all peripherals attached to your device. You can click the settings icons to view the properties of each.



Networking

The Networking page manages network connections on the device. Unless you are connected to Device Portal through USB, changing these settings will likely disconnect you from Device Portal.

- **Available networks:** Shows the WiFi networks available to the device. Clicking or tapping on a network will allow you to connect to it and supply a passkey if needed. Device Portal does not yet support Enterprise Authentication. You can also use the **Profiles** dropdown to attempt to connect to any of the WiFi profiles known to the device.
- **IP configuration:** Shows address information about each of the host device's network ports.



Service features and notes

DNS-SD

Device Portal advertises its presence on the local network using DNS-SD. All Device Portal instances, regardless of their device type, advertise under "WDP._wdp._tcp.local". The TXT records for the service instance provide the following:

Key	Type	Description
S	int	Secure port for Device Portal. If 0 (zero), Device Portal is not listening for HTTPS connections.
D	string	Type of device. This will be in the format "Windows.*", for example, Windows.Xbox or Windows.Desktop
A	string	Device architecture. This will be Arm, x86, or AMD64.
T	null-character delineated list of strings	User-applied tags for the device. See the Tags REST API for how to use this. List is double-null terminated.

Connecting on the HTTPS port is suggested, as not all devices are listening on the HTTP port advertised by the DNS-SD record.

CSRF Protection and Scripting

In order to protect against [CSRF attacks](#), a unique token is required on all non-GET requests. This token, the X-CSRF-Token request header, is derived from a session cookie,

CSRF-Token. In the Device Portal web UI, the CSRF-Token cookie is copied into the X-CSRF-Token header on each request.

Important

This protection prevents usages of the REST APIs from a standalone client (such as command-line utilities). This can be solved in 3 ways:

- Use an "auto-" username. Clients that prepend "auto-" to their username will bypass CSRF protection. It is important that this username not be used to log in to Device Portal through the browser, as it will open up the service to CSRF attacks. Example: If Device Portal's username is "admin", `curl -u auto-admin:password <args>` should be used to bypass CSRF protection.
- Implement the cookie-to-header scheme in the client. This requires a GET request to establish the session cookie, and then the inclusion of both the header and the cookie on all subsequent requests.
- Disable authentication and use HTTP. CSRF protection only applies to HTTPS endpoints, so connections on HTTP endpoints will not need to do either of the above.

Cross-Site WebSocket Hijacking (CSWSH) protection

To protect against [CSWSH attacks](#), all clients opening a WebSocket connection to Device Portal must also provide an Origin header that matches the Host header. This proves to Device Portal that the request comes either from the Device Portal UI or a valid client application. Without the Origin header your request will be rejected.

See also

[Device Portal core API reference](#)

Windows Device Portal for Desktop

Article • 10/20/2022

Windows Device Portal (WDP) is a device management and debugging tool that lets you configure and manage device settings and view diagnostic information over HTTP from a web browser. For WDP details on other devices, see [Windows Device Portal overview](#).

You can use the WDP for the following:

- Manage device settings (similar to the **Windows Settings** app)
- See and manipulate a list of running processes
- Install, delete, launch, and terminate apps
- Change Wi-Fi profiles, view signal strength, and see ipconfig details
- View live graphs of CPU, memory, I/O, network, and GPU usage
- Collect process dumps
- Collect ETW traces
- Manipulate the isolated storage of sideloaded apps

Set up Windows Device Portal on a Desktop device

Turn on developer mode

Starting in Windows 10, version 1607, some of the newer features for desktop are only available when developer mode is enabled. For information about how to enable developer mode, see [Enable your device for development](#).

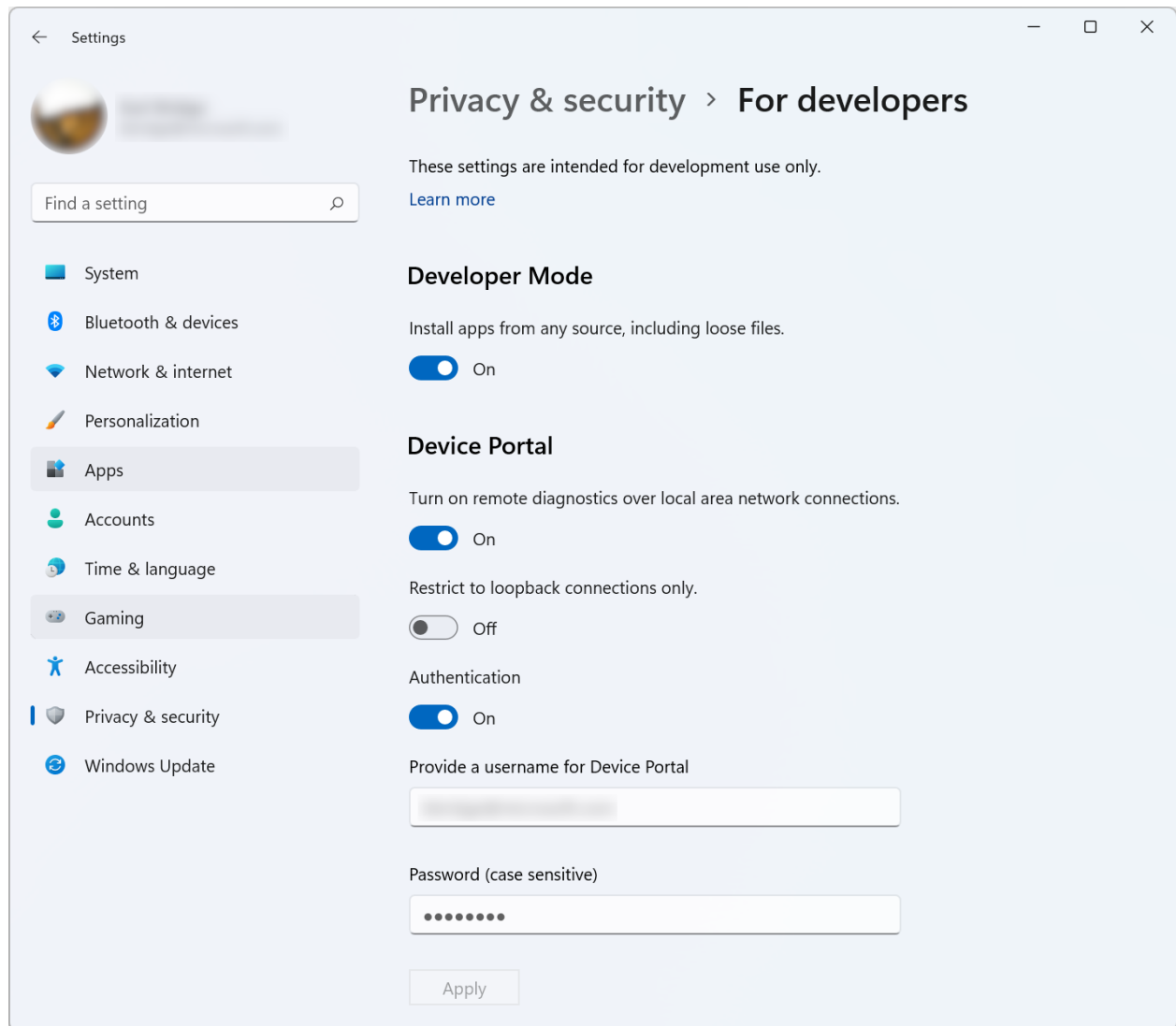
Important

Sometimes, due to network or compatibility issues, developer mode won't install correctly on your device. See the [relevant section of Enable your device for development](#) for help troubleshooting these issues.

Turn on Windows Device Portal

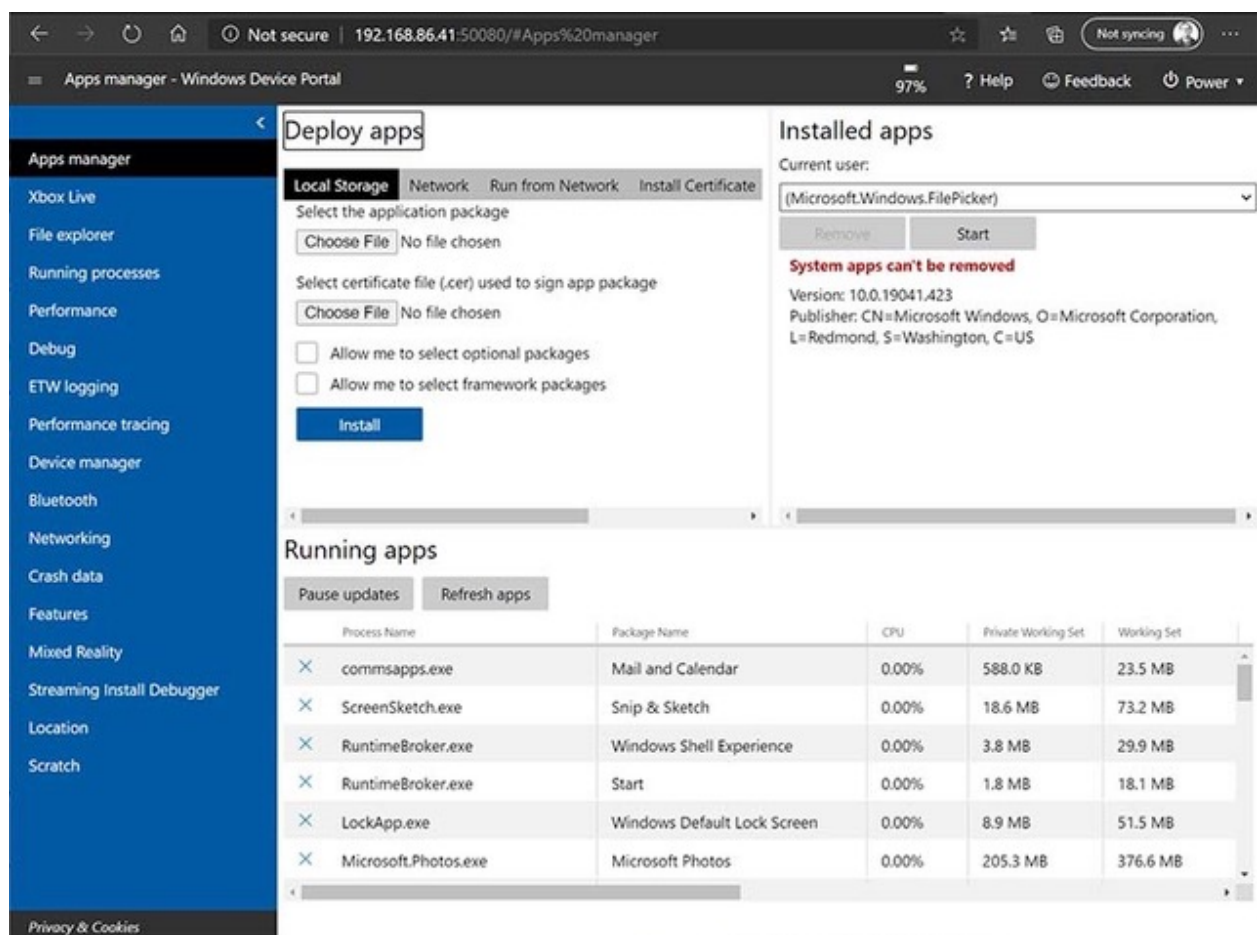
You can enable WDP in the **For developers** section of **Settings** (Settings -> Privacy & security -> For developers). When you enable it, you must also create a corresponding

username and password. Do not use your Microsoft account or other Windows credentials.



Once WDP is enabled, you will see web links at the bottom of the section. Take note of the port number appended to the end of the listed URLs: this number is randomly generated when WDP is enabled but should remain consistent between reboots of the desktop.

These links offer two ways to connect to WDP: over the local network (including VPN) or through the local host. Once you connect, it should look something like this:



Turn off Windows Device Portal

You can disable WDP in the **For developers** section of **Windows Settings**.

Connect to Windows Device Portal

To connect through local host, open a browser window and enter the one of the URIs shown here (based on the connection type you're using).

- Localhost: `http://127.0.0.1:<PORT>` or `http://localhost:<PORT>`
- Local Network: `https://<IP address of the desktop>:<PORT>`

HTTPS is required for authentication and secure communication.

If you are using WDP in a protected environment, for example, in a test lab, in which you trust everyone on your local network, have no personal information on the device, and have unique requirements, you can disable the Authentication option. This enables unencrypted communication, and allows anyone with the IP address of your computer to connect to and control it.

Windows Device Portal content

WDP provides the following set of pages.

- Apps manager
- Xbox Live
- File explorer
- Running Processes
- Performance
- Debug
- ETW (Event Tracing for Windows) logging
- Performance tracing
- Device manager
- Bluetooth
- Networking
- Crash data
- Features
- Mixed Reality
- Streaming Install Debugger
- Location
- Scratch

Using Windows Device Portal to test and debug MSIX apps

The following video describes how to test and debug [MSIX apps](#) remotely using Windows Device Portal.

<https://www.youtube-nocookie.com/embed/PdgXeOMt4hk> [↗](#)

More Windows Device Portal options

The following sections provide details on other Windows Device Portal options.

Registry-based configuration

If you would like to select port numbers for WDP (such as 80 and 443), you can set the following regkeys:

- Under

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\WebManagement\Service
```


- `UseDynamicPorts`: A required DWORD. Set this to 0 in order to retain the port numbers you've chosen.
- `HttpPort`: A required DWORD. Contains the port number on which WDP listens for HTTP connections.
- `HttpsPort`: A required DWORD. Contains the port number on which WDP listens for HTTPS connections.

Under the same regkey path, you can also turn off the authentication requirement:

- `UseDefaultAuthorizer` - 0 for disabled, 1 for enabled.
 - This controls both the basic auth requirement for each connection and the redirect from HTTP to HTTPS.

Command line options for Windows Device Portal

From an administrative command prompt, you can enable and configure parts of WDP. To see the latest set of commands supported on your build, you can run `webmanagement /?`

- `sc start webmanagement` or `sc stop webmanagement`
 - Turn the service on or off. This still requires developer mode to be enabled.
- `-Credentials <username> <password>`
 - Set a username and password for WDP. The username must conform to Basic Auth standards, so cannot contain a colon (:) and should be built out of standard ASCII characters for example, [a-zA-Z0-9] as browsers do not parse the full character set in a standard way.
- `-DeleteSSL`
 - This resets the SSL certificate cache used for HTTPS connections. If you encounter TLS connection errors that cannot be bypassed (as opposed to the expected certificate warning), this option may fix the problem for you.
- `-SetCert <pfxPath> <pfxPassword>`
 - See [Provisioning Windows Device Portal with a custom SSL certificate](#) for details.
 - This allows you to install your own SSL certificate to fix the SSL warning page that is typically seen in WDP.
- `-Debug <various options for authentication, port selection, and tracing level>`
 - Run a standalone version of WDP with a specific configuration and visible debug messages. This is most useful for building a [packaged plugin](#).
 - See the [MSDN Magazine article](#) for details on how to run this as System to fully test your packaged plugin.

Troubleshooting

Below are some common errors that you may encounter when setting up Windows Device Portal.

WindowsUpdateSearch returns invalid number of updates (0x800f0950)

You may get this error when trying to install the developer packages on a pre-release build of Windows. These Feature-on-Demand (FoD) packages are hosted on Windows Update, and downloading them on pre-release builds requires that you opt into flighting. If your installation is not opted into flighting for the right build and ring combination, the payload will not be downloadable. Double-check the following:

1. Navigate to **Settings > Update & Security > Windows Insider Program** and confirm that the **Windows Insider account** section has your correct account info. If you don't see that section, select **Link a Windows Insider account**, add your email account, and confirm that it shows up under the **Windows Insider account** heading (you may need to select **Link a Windows Insider account** a second time to actually link a newly added account).
2. Under **What kind of content would you like to receive?**, make sure **Active development of Windows** is selected.
3. Under **What pace do you want to get new builds?**, make sure **Windows Insider Fast** is selected.
4. You should now be able to install the FoDs. If you've confirmed that you're on Windows Insider Fast and still cannot install the FoDs, please provide feedback and attach the log files under **C:\Windows\Logs\CBS**.

[SC] StartService: OpenService FAILED 1060: The specified service does not exist as an installed service

You may get this error if the developer packages aren't installed. Without the developer packages, there is no web management service. Try installing the developer packages again.

CBS cannot start download because the system is on metered network (CBS_E_METERED_NETWORK)

You may get this error if you're on a metered internet connection. You won't be able to download the developer packages on a metered connection.

See also

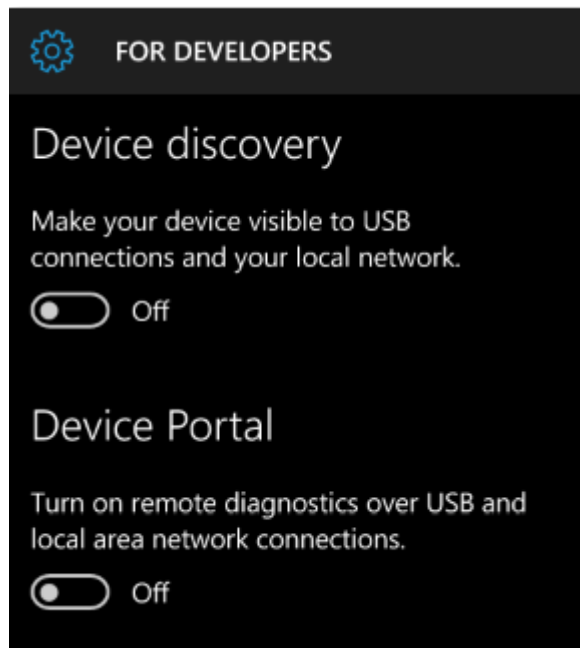
- [Windows Device Portal overview](#)
- [Windows Device Portal core API reference](#)

Windows Device Portal for Mobile

Article • 10/20/2022

Starting in Windows 10, Version 1511, additional developer features are available for the mobile device family. These features are available only when Developer mode is enabled on the device.

For info about how to enable Developer mode, see [Enable your device for development](#).

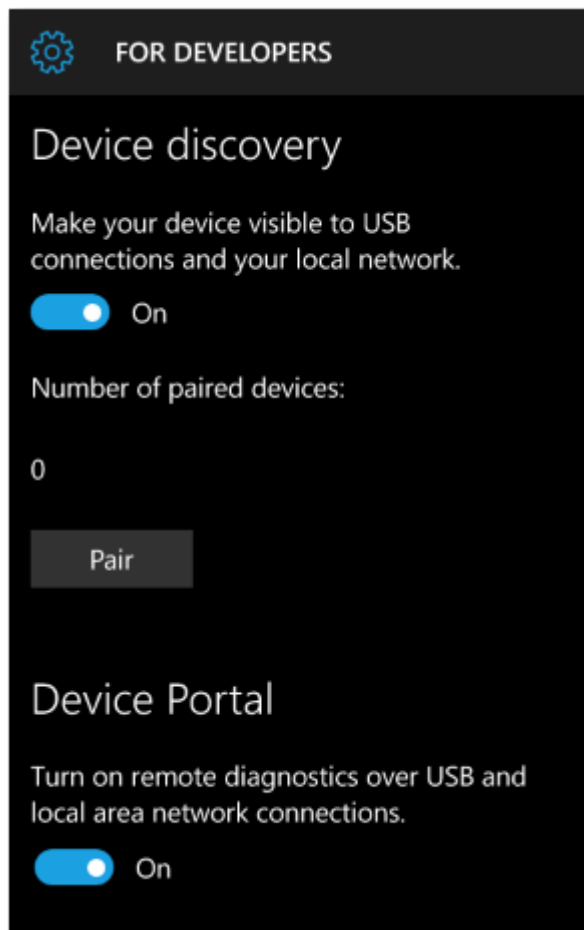


Set up Windows Device Portal on Windows Phone

Turn on device discovery and pairing

To connect to Windows Device Portal (WDP), you must enable Device discovery and Device Portal in your phone's settings. This lets you pair your phone with a PC or other Windows 10 device. Both devices must be connected to the same subnet of the network by a wired or wireless connection, or they must be connected by USB.

The first time you connect to Device Portal, you are asked for a case-sensitive, 6 character security code. This ensures that you have access to the phone, and keeps you safe from attackers. Press the Pair button on your phone to generate and display the code, then enter the 6 characters into the text box in the browser.



You can choose from 3 ways to connect to Device Portal: USB, local host, and over the local network (including VPN and tethering).

To connect to Device Portal

1. In your browser, enter the address shown here for the connection type you're using.

- USB: `http://127.0.0.1:10080`

Use this address when the phone is connected to a PC via a USB connection. Both devices must have Windows 10, Version 1511 or later.

- Localhost: `http://127.0.0.1`

Use this address to view Device Portal locally on the phone in Microsoft Edge for Windows 10 Mobile.

- Local Network: `https://<The IP address or hostname of the phone>`

Use this address to connect over a local network.

The IP address of the phone is shown in the Device Portal settings on the phone. HTTPS is required for authentication and secure communication. The hostname